

```
01 revenue = 2_750_000
02 costs = revenue * 0.62
03 for month in range(1, 13):
04     close_books(month)
05 if stock < reorder_level:
06     place_order(supplier)
```

COMPANION BOOKLET

Python

Script Booklet

Every script from Python Foundations for Business, with a short explanation, the libraries it needs, and the output you should see.

138

scripts

16

chapters + answers

4

libraries to install

Contents

START HERE	Setting up Python and the required libraries		4
CHAPTER 1	Python and the Business of Information	5 scripts	6
CHAPTER 2	Variables, Data Types and Operators	10 scripts	8
CHAPTER 3	Working with Text (Strings)	8 scripts	13
CHAPTER 4	Input, Output and Formatting	7 scripts	17
CHAPTER 5	Making Decisions with Conditionals	10 scripts	21
CHAPTER 6	Repetition with Loops	10 scripts	26
CHAPTER 7	Lists and Tuples	9 scripts	31
CHAPTER 8	Dictionaries and Sets	9 scripts	36
CHAPTER 9	Functions	10 scripts	40
CHAPTER 10	Modules and the Standard Library	7 scripts	45
CHAPTER 11	Working with Files	7 scripts	49
CHAPTER 12	Errors, Exceptions and Debugging	8 scripts	54
CHAPTER 13	Object-Oriented Programming Basics	5 scripts	58
CHAPTER 14	Introduction to Data Analysis with pandas	11 scripts	63
CHAPTER 15	Visualizing Business Data with Matplotlib	6 scripts	72
CHAPTER 16	Capstone Project: Inventory and Sales Manager	5 scripts	79
APPENDIX C	Answers to Selected Exercises	11 scripts	90
HELP	Troubleshooting common problems		96

Downloads that go with this booklet: the `python_scripts` folder contains every script below as a runnable `.py` file, the sample data files, and `requirements.txt`.

How to use this booklet

This booklet collects every code example from the textbook in one place, in chapter order. Each script appears on a card like this:

- **Script number and title** match the chapter and topic in the book.
- **File name:** the same script as a ready-to-run `.py` file in the `python_scripts` download folder.
- **Requirement chips:** `Built-in only` needs nothing extra, `Standard library` ships with Python (just `import` it), and `pip install ...` must be installed once.
- **Setup blocks** (shaded paper colour) are a few lines added so a script that continues an earlier example can run on its own.
- **Output** is what you should see. It was produced by actually running every script. In sample sessions, **highlighted text** is what the user typed.

Study tip. Type the scripts rather than copying them. When a script works, change one value and predict the new output before you run it again.

Sample data files (included in the download)

`sales.csv` (Scripts 11.3, 11.6, 12.4, 14.2), `sales.xlsx` with a sheet named *September* (14.2), and `exports/branch_*.csv` (11.7). Keep them in the same folder as the scripts that use them.

SALES.CSV

```
date,region,product,units,unit_price
2026-09-01,North,Laptop,3,950
2026-09-01,South,Mouse,25,18
2026-09-02,North,Monitor,4,210
2026-09-02,East,Laptop,2,950
2026-09-03,South,Laptop,1,950
```

Start here: setting up

1 · Install Python 3.10 or newer

Download Python from python.org/downloads and run the installer. On Windows, **tick “Add python.exe to PATH”** on the first installer screen. Version 3.10 or newer is required because some scripts use `match` (Scripts 5.9 and 16.5).

Easier alternative: the **Anaconda** distribution (anaconda.com/download) installs Python together with pandas, numpy, matplotlib, openpyxl and Jupyter in one step.
No installation at all: Google Colab (colab.research.google.com) runs in the browser and already has every library in this booklet.

2 · Check the installation

TERMINAL / COMMAND PROMPT

```
python --version      # Windows (or: py --version)
python3 --version     # macOS / Linux
```

3 · Install the four third-party libraries (once)

Where you work	Command
Windows	<code>py -m pip install pandas numpy matplotlib openpyxl</code>
macOS / Linux	<code>python3 -m pip install pandas numpy matplotlib openpyxl</code>
Jupyter or Colab cell	<code>%pip install pandas numpy matplotlib openpyxl</code>
Anaconda Prompt	<code>conda install pandas numpy matplotlib openpyxl</code> (usually already installed)
Using the download	<code>pip install -r requirements.txt</code> (run it inside the <code>python_scripts</code> folder)

4 · Confirm the libraries work

```
import pandas, numpy, matplotlib, openpyxl
print("pandas", pandas.__version__)
print("matplotlib", matplotlib.__version__)
print("All libraries ready!")
```

5 · Run a script

Open a terminal **in the folder that contains the script** and type `python file_name.py` (Windows: `py file_name.py`, macOS: `python3 file_name.py`). In VS Code, press the ► Run button. In Jupyter or Colab, paste the code into a cell and press Shift + Enter.

Which libraries does each script need?

Library	Type	How to get it	What it does	Used in scripts
Built into Python: nothing to install or import				
<code>print, input, len, sum, ...</code>	Built-in	Always available	Core functions and types	Chapters 1–9, 12, 13
Standard library: comes with Python, only needs an <code>import</code> line				
<code>decimal</code>	Standard	<code>from decimal import Decimal</code>	Exact money arithmetic	2.9
<code>math</code>	Standard	<code>import math</code>	ceil, floor, sqrt, exp	10.1, 10.2
<code>statistics</code>	Standard	<code>import statistics</code>	mean, median, stdev	10.1, 10.3
<code>random</code>	Standard	<code>import random</code>	Random numbers and simulation	10.4, 10.5
<code>datetime</code>	Standard	<code>from datetime import ...</code>	Dates, due dates, timestamps	10.6, 11.2, 16.1, 16.2, 16.5
<code>csv</code>	Standard	<code>import csv</code>	Read and write CSV files	11.3, 11.4, 11.7, 16.1, 16.2, 16.5
<code>json</code>	Standard	<code>import json</code>	Save and load settings and records	11.5, 16.1, 16.2, 16.5
<code>pathlib</code>	Standard	<code>from pathlib import Path</code>	Files and folders	11.6, 11.7, 16.1, 16.2, 16.5
Third-party: install once with pip (see the previous page)				
<code>pandas</code>	pip	<code>pip install pandas</code>	Tables (DataFrames), analysis	14.1, 14.2, 14.3, 14.4, 14.5, 14.6, 14.7, 14.8, 14.9, 14.10, 14.11, 15.5, 16.4, 16.5, C.11
<code>numpy</code>	pip	<code>pip install numpy (installed with pandas)</code>	Numbers and <code>np.nan</code>	14.9
<code>matplotlib</code>	pip	<code>pip install matplotlib</code>	Charts	15.1, 15.2, 15.3, 15.4, 15.5, 15.6
<code>openpyxl</code>	pip	<code>pip install openpyxl</code>	Lets pandas read and write <code>.xlsx</code> files (never imported directly)	14.2, 14.11, 16.4, 16.5

CHAPTER 1

Python and the Business of Information

Your first programs: printing text, using Python as a calculator, comments, and reading an error message.

Libraries used: none: built-in Python only

Script 1.1 Hello, Business World

ch01/s1_01_hello_business_world.py **Built-in only · nothing to install**

The classic first program. `print()` displays the text inside the quotation marks (a string).

```
print("Hello, Business World!")
```

OUTPUT

```
Hello, Business World!
```

Script 1.2 A three-line status report

ch01/s1_02_a_three_line_status_report.py **Built-in only · nothing to install**

Python runs lines top to bottom (sequential execution), so the three lines print in order.

```
print("Company: Cedar Trading Co.")  
print("Report: Daily Sales Summary")  
print("Status: All systems operational")
```

OUTPUT

```
Company: Cedar Trading Co.  
Report: Daily Sales Summary  
Status: All systems operational
```

Script 1.3 The interpreter as a calculatorch01/s1_03_the_interpreter_as_a_calculator.py **Built-in only · nothing to install**

Type `python` (or `python3`) in a terminal to get the `>>>` prompt, then type expressions. The last line is a percentage change from 95 to 120. Type `exit()` to leave.

INTERACTIVE INTERPRETER (TYPE AFTER `>>>`)

```
>>> 1250 + 780
2030
>>> 45000 * 0.11
4950.0
>>> (120 - 95) / 95 * 100
26.31578947368421
```

Script 1.4 Comments that explain a calculationch01/s1_04_comments_that_explain_a_calculation.py **Built-in only · nothing to install**

Everything after `#` is a note for humans; Python ignores it. Good comments record the business rule behind the code.

```
# Calculate the total price of an order
unit_price = 25      # price per unit in USD
quantity = 12        # number of units ordered
print(unit_price * quantity)
```

OUTPUT

300

Script 1.5 Reading a syntax errorch01/s1_05_reading_a_syntax_error.py **Built-in only · nothing to install**

A deliberately broken line. The error message tells you exactly what is wrong: an opening parenthesis was never closed.

```
print("Quarterly revenue report"
```

ERROR MESSAGE (EXPECTED)

```
File "s1_05_reading_a_syntax_error.py", line 1
  print("Quarterly revenue report"
    ^
SyntaxError: '(' was never closed
```

CHAPTER 2

Variables, Data Types and Operators

Storing values in named variables, the four core types (int, float, str, bool), conversions, and arithmetic, comparison and logical operators.

Libraries used: decimal

Script 2.1 Creating variables

ch02/s2_01_creating_variables.py **Built-in only · nothing to install**

The = sign stores the value on the right under the name on the left. Underscores in 2_750_000.50 are only for readability.

```
company_name = "Cedar Trading Co."  
employees = 48  
annual_revenue = 2_750_000.50  
is_listed = False  
  
print(company_name)  
print(employees)  
print(annual_revenue)  
print(is_listed)
```

OUTPUT

```
Cedar Trading Co.  
48  
2750000.5  
False
```

Script 2.2 Updating a variable

ch02/s2_02 Updating a variable.py **Built-in only · nothing to install**

A variable can be reassigned using its own old value: here 15 units are subtracted from stock.

```
stock = 100  
stock = stock - 15      # 15 units were sold  
print(stock)
```

OUTPUT

```
85
```

Script 2.3 Checking data types

ch02/s2_03_checking_data_types.py Built-in only · nothing to install

`type()` reveals whether a value is an int, float, str or bool.

```
print(type(48))
print(type(19.99))
print(type("INV-2031"))
print(type(True))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Script 2.4 Type conversion

ch02/s2_04_type_conversion.py Built-in only · nothing to install

Text that looks like a number must be converted with `int()` or `float()` before doing arithmetic; `str()` turns a number back into text.

```
quantity_text = "25"
price_text = "12.50"

quantity = int(quantity_text)
price = float(price_text)
total = quantity * price

print(total)
print("Total: " + str(total))
```

OUTPUT

```
312.5
Total: 312.5
```

Script 2.5 Floor division and remainder

ch02/s2_05_floor_division_and_remainder.py **Built-in only · nothing to install**

// gives whole boxes (floor division) and % gives the units left over (modulo). Useful for packaging and batching.

```
units_ordered = 150
box_size = 12

full_boxes = units_ordered // box_size
leftover = units_ordered % box_size

print("Full boxes:", full_boxes)
print("Loose units:", leftover)
```

OUTPUT

```
Full boxes: 12
Loose units: 6
```

Script 2.6 Business example: compound interest

ch02/s2_06_compound_interest.py **Built-in only · nothing to install**

Future value = principal $\times$ (1 + rate) ** years. ** is the power operator; round() tidies the result to 2 decimals.

```
principal = 10_000          # initial investment
rate = 0.06                 # 6% annual interest
years = 5

future_value = principal * (1 + rate) ** years
interest_earned = future_value - principal

print("Future value:", round(future_value, 2))
print("Interest earned:", round(interest_earned, 2))
```

OUTPUT

```
Future value: 13382.26
Interest earned: 3382.26
```

Script 2.7 Augmented assignment

ch02/s2_07_augmented_assignment.py **Built-in only · nothing to install**

`+=`, `-=` and `*=` update a variable in place: a deposit, a withdrawal, then 1% interest.

```
balance = 5000
balance += 1200 # deposit, same as balance = balance + 1200
balance -= 350 # withdrawal
balance *= 1.01 # 1% interest
print(round(balance, 2))
```

OUTPUT

```
5908.5
```

Script 2.8 Logical operators: loan eligibility

ch02/s2_08_logical_operators_loan_eligibility.py **Built-in only · nothing to install**

and requires every condition to be True; not flips a Boolean. The result is a single True/False decision.

```
credit_score = 710
annual_income = 42_000
has_defaults = False

eligible = (credit_score >= 680 and annual_income >= 30_000
           and not has_defaults)
print("Loan eligible:", eligible)
```

OUTPUT

```
Loan eligible: True
```

Script 2.9 Floating-point precision and Decimal

ch02/s2_09_floating_point_precision_and_decimal.py **Standard library · decimal**

Floats cannot store 0.1 exactly, so tiny errors appear. For money where exact cents matter, use the Decimal type from the standard library.

```
print(0.1 + 0.2)

from decimal import Decimal
print(Decimal("0.10") + Decimal("0.20"))
```

OUTPUT

```
0.30000000000000004
0.30
```

Script 2.10 Business case: break-even calculatorch02/s2_10_break_even_calculator.py **Built-in only · nothing to install**

Contribution margin = price – variable cost; break-even units = fixed costs ÷ contribution margin.

```
price_per_cup = 3.50
variable_cost_per_cup = 1.20
fixed_costs = 6_900

contribution_margin = price_per_cup - variable_cost_per_cup
break_even_units = fixed_costs / contribution_margin

print("Contribution margin per cup:", contribution_margin)
print("Break-even cups per month:", round(break_even_units))
print("Break-even cups per day (30 days):", round(break_even_units / 30))
```

OUTPUT

```
Contribution margin per cup: 2.3
Break-even cups per month: 3000
Break-even cups per day (30 days): 100
```

CHAPTER 3

Working with Text (Strings)

Creating, joining, slicing, cleaning, searching and splitting text: the everyday tools of data cleaning.

Libraries used: none: built-in Python only

Script 3.1 Creating strings

ch03/s3_01_creating_strings.py **Built-in only · nothing to install**

Strings can use single, double or triple quotes. Triple quotes allow text across several lines.

```
product = "Wireless Mouse"
code = 'WM-204'
policy = """Returns are accepted within 30 days
of purchase with a valid receipt."""

print(product)
print(code)
print(policy)
```

OUTPUT

```
Wireless Mouse
WM-204
Returns are accepted within 30 days
of purchase with a valid receipt.
```

Script 3.2 Joining and repeating strings

ch03/s3_02_joining_and_repeating_strings.py **Built-in only · nothing to install**

`+` joins (concatenates) strings; `*` repeats a string, handy for divider lines.

```
first = "Maya"
last = "Haddad"
full_name = first + " " + last
print(full_name)
print("-" * 30)
```

OUTPUT

```
Maya Haddad
-----
```

Script 3.3 Indexing, slicing and len()ch03/s3_03_indexing_slicing_and_len.py **Built-in only · nothing to install**

Positions start at 0; negative indexes count from the end. [start:end] slices up to, but not including, end.

```
invoice = "INV-2026-00457"

print(invoice[0])           # first character
print(invoice[-1])          # last character
print(invoice[0:3])          # characters 0, 1, 2
print(invoice[4:8])          # the year
print(invoice[-5:])          # last five characters

print(len(invoice))
```

OUTPUT

```
I
7
INV
2026
00457
14
```

Script 3.4 Strings are immutablech03/s3_04_strings_are_immutable.py **Built-in only · nothing to install**

String methods return a new string; the original never changes. Store the result if you want to keep it.

```
city = "beirut"
city.upper()           # returns "BEIRUT" but the result is discarded
print(city)
city = city.upper()     # store the new string
print(city)
```

OUTPUT

```
beirut
BEIRUT
```

Script 3.5 Business example: cleaning customer datach03/s3_05_cleaning_customer_data.py **Built-in only · nothing to install**

split() + join() collapses extra spaces, title() fixes capitals, strip()/lower() normalise emails, replace() removes characters.

```
raw_name = " mAYA haddad "  
raw_email = " Maya.Haddad@Example.COM "  
raw_phone = "+961 3-123 456"  
  
clean_name = " ".join(raw_name.split()).title()  
clean_email = raw_email.strip().lower()  
clean_phone = raw_phone.replace(" ", "").replace("-", "")  
  
print(clean_name)  
print(clean_email)  
print(clean_phone)
```

OUTPUT

```
Maya Haddad  
maya.haddad@example.com  
+9613123456
```

Script 3.6 Searching textch03/s3_06_searching_text.py **Built-in only · nothing to install**

The in operator tests whether text appears; count() counts occurrences (after lower()) so case does not matter).

```
feedback = "The delivery was late but the product quality is excellent."  
  
print("late" in feedback)  
print("refund" in feedback)  
print(feedback.lower().count("the"))
```

OUTPUT

```
True  
False  
2
```

Script 3.7 Splitting structured textch03/s3_07_splitting_structured_text.py **Built-in only · nothing to install**

`split(",")` breaks a comma-separated record into a list of fields, which you then convert to the right types.

```
record = "C1043,Rami Khoury,Gold,2150.75"
fields = record.split(",")

customer_id = fields[0]
name = fields[1]
tier = fields[2]
total_spent = float(fields[3])

print(name, "is a", tier, "customer who has spent", total_spent)
```

OUTPUT

```
Rami Khoury is a Gold customer who has spent 2150.75
```

Script 3.8 Business case: validating product codesch03/s3_08_validating_product_codes.py **Built-in only · nothing to install**

Checks the pattern ABC-1234 with slicing plus `isalpha()`, `isupper()` and `isdigit()`. Wrapping the conditions in `()` lets them span lines.

```
code = "LAP-0932"

valid = (
    len(code) == 8
    and code[:3].isalpha()
    and code[:3].isupper()
    and code[3] == "-"
    and code[4:].isdigit()
)

print(code, "is valid:", valid)
```

OUTPUT

```
LAP-0932 is valid: True
```

CHAPTER 4

Input, Output and Formatting

Reading keyboard input, controlling `print()`, and producing professional number formats and aligned tables with f-strings.

Libraries used: none: built-in Python only

Script 4.1 Getting input from the user

ch04/s4_01_getting_input_from_the_user.py **Built-in only · nothing to install**

`input()` shows a prompt and always returns text (a string).

```
name = input("Enter your name: ")
print("Welcome,", name)
```

SAMPLE SESSION · HIGHLIGHTED = TYPED BY THE USER

```
Enter your name: Lina
Welcome, Lina
```

Script 4.2 Converting numeric input

ch04/s4_02_converting_numeric_input.py **Built-in only · nothing to install**

Wrap `input()` in `float()` or `int()` so the value can be used in calculations.

```
price = float(input("Unit price: "))
quantity = int(input("Quantity: "))
print("Total:", price * quantity)
```

SAMPLE SESSION · HIGHLIGHTED = TYPED BY THE USER

```
Unit price: 12.5
Quantity: 8
Total: 100.0
```

Script 4.3 `print()` options: `sep` and `end`

ch04/s4_03_print_options_sep_and_end.py **Built-in only · nothing to install**

`sep` sets the separator between items; `end` replaces the default newline at the end of the line.

```
print("Jan", "Feb", "Mar", sep=" | ")
print("Loading", end="...")
print("done")
```

OUTPUT

```
Jan | Feb | Mar  
Loading...done
```

Script 4.4 f-stringsch04/s4_04_f_strings.py **Built-in only · nothing to install**

Put `f` before the quotes and any variable or expression inside `{ }` is inserted into the text.

```
client = "Atlas Logistics"  
invoices = 14  
balance = 18250.5  
  
print(f"{client} has {invoices} open invoices totalling {balance} USD.")  
  
price = 80  
discount = 0.15  
print(f"Discounted price: {price * (1 - discount)}")
```

OUTPUT

```
Atlas Logistics has 14 open invoices totalling 18250.5 USD.  
Discounted price: 68.0
```

Script 4.5 Formatting numbersch04/s4_05_formatting_numbers.py **Built-in only · nothing to install**

`:.2f` adds thousands separators and 2 decimals, `:.1%` shows a percentage, `:06d` pads with zeros.

```
revenue = 1_845_320.456  
growth = 0.0735  
order_no = 57  
  
print(f"Revenue: ${revenue:,.2f}")  
print(f"Growth: {growth:.1%}")  
print(f"Order: ORD-{order_no:06d}")
```

OUTPUT

```
Revenue: $1,845,320.46  
Growth: 7.3%  
Order: ORD-000057
```

Script 4.6 Producing aligned tables

ch04/s4_06_producing_aligned_tables.py **Built-in only · nothing to install**

:<10 left-aligns and :>8 right-aligns in a fixed width, so columns line up.

```
print(f"{'Region':<10}{'Units':>8}{'Revenue':>14}")
print("-" * 32)
print(f"{'North':<10}{'1240':>8}{'186000':>14,.2f}")
print(f"{'South':<10}{'985':>8}{'147750':>14,.2f}")
print(f"{'East':<10}{'1502':>8}{'225300':>14,.2f}")
```

OUTPUT

Region	Units	Revenue
North	1240	186,000.00
South	985	147,750.00
East	1502	225,300.00

Script 4.7 Business case: a sales receipt

ch04/s4_07_a_sales_receipt.py **Built-in only · nothing to install**

Combines input, VAT arithmetic and alignment (:^34 centres the title) to print a formatted receipt.

```
item = input("Item: ")
price = float(input("Unit price: "))
qty = int(input("Quantity: "))

VAT_RATE = 0.11
subtotal = price * qty
vat = subtotal * VAT_RATE
total = subtotal + vat

print()
print(f"{'CEDAR TRADING CO.':^34}")
print("=" * 34)
print(f"{'item':<20}{'qty':>4} x {'price':>7.2f}")
print("-" * 34)
print(f"{'Subtotal':<20}{'subtotal':>14,.2f}")
print(f"{'VAT 11%':<20}{'vat':>14,.2f}")
print(f"{'TOTAL':<20}{'total':>14,.2f}")
print("=" * 34)
```

SAMPLE SESSION · **HIGHLIGHTED** = TYPED BY THE USER

Item: **USB-C Hub**

Unit price: **34.90**

Quantity: **3**

CEDAR TRADING CO.

=====

USB-C Hub	3 x	34.90
-----------	-----	-------

Subtotal	104.70
----------	--------

VAT 11%	11.52
---------	-------

TOTAL	116.22
-------	--------

=====

CHAPTER 5

Making Decisions with Conditionals

Selection: running code only when a condition is true, with if, elif, else, nested decisions, the conditional expression and match.

Libraries used: none: built-in Python only

Script 5.1 The if statement

ch05/s5_01_the_if_statement.py Built-in only · nothing to install

The indented line runs only when the condition is True; the unindented line always runs.

```
order_total = 120

if order_total >= 100:
    print("Free shipping applied.")
print("Thank you for your order.")
```

OUTPUT

```
Free shipping applied.
Thank you for your order.
```

Script 5.2 if-else: reorder alert

ch05/s5_02_if_else_reorder_alert.py Built-in only · nothing to install

else provides the alternative path when the condition is False.

```
stock = 3
reorder_level = 10

if stock < reorder_level:
    print("Stock is low. Place a reorder.")
else:
    print("Stock level is sufficient.")
```

OUTPUT

```
Stock is low. Place a reorder.
```

Script 5.3 if-elif-else: customer tiersch05/s5_03_if_elif_else_customer_tiers.py **Built-in only · nothing to install**

Conditions are checked top to bottom and only the first True branch runs, so order thresholds from highest to lowest.

```
annual_spend = 3400

if annual_spend >= 5000:
    tier = "Platinum"
elif annual_spend >= 2500:
    tier = "Gold"
elif annual_spend >= 1000:
    tier = "Silver"
else:
    tier = "Bronze"

print(f"Customer tier: {tier}")
```

OUTPUT

Customer tier: Gold

Script 5.4 Combining conditionsch05/s5_04_combining_conditions.py **Built-in only · nothing to install**

and / or / not build compound rules, such as account activation requirements.

```
age = 27
has_id = True
account_balance = 150

if age >= 18 and has_id and account_balance > 0:
    print("Account can be activated.")
else:
    print("Activation requirements not met.")
```

OUTPUT

Account can be activated.

Script 5.5 Chained comparisonsch05/s5_05_chained_comparisons.py **Built-in only · nothing to install**

70 <= score < 80 is Python shorthand for 70 <= score and score < 80.

```
score = 74
if 70 <= score < 80:
    print("Grade: C")
```

OUTPUT

Grade: C

Script 5.6 Nested conditions

ch05/s5_06_nested_conditions.py **Built-in only · nothing to install**

An if inside another if: members get a discount that depends on order size.

```
is_member = True
order_total = 60

if is_member:
    if order_total >= 50:
        discount = 0.10
    else:
        discount = 0.05
else:
    discount = 0.0

print(f"Discount rate: {discount:.0%}")
```

OUTPUT

Discount rate: 10%

Script 5.7 Truthiness

ch05/s5_07_truthiness.py **Built-in only · nothing to install**

Empty strings, 0, empty lists and None count as False, so if not customer_email detects a missing value.

```
customer_email = ""

if not customer_email:
    print("Warning: email address is missing.")
```

OUTPUT

Warning: email address is missing.

Script 5.8 The conditional expression

ch05/s5_08_the_conditional_expression.py Built-in only · nothing to install

A one-line if/else that picks one of two values: `value_if_true` if condition else `value_if_false`.

```
balance = -40
status = "Overdrawn" if balance < 0 else "In good standing"
print(status)
```

OUTPUT

```
Overdrawn
```

Script 5.9 Pattern matching with match

ch05/s5_09_pattern_matching_with_match.py Built-in only · nothing to install Python 3.10+

`match` compares one value against several case patterns; `case _` is the catch-all. Requires Python 3.10 or newer.

```
status_code = "SHP"

match status_code:
    case "NEW":
        message = "Order received"
    case "PRO":
        message = "Order being processed"
    case "SHP":
        message = "Order shipped"
    case "DEL":
        message = "Order delivered"
    case _:
        message = "Unknown status"

print(message)
```

OUTPUT

```
Order shipped
```

Script 5.10 Business case: progressive income taxch05/s5_10_progressive_income_tax.py **Built-in only · nothing to install**

Each income band is taxed at its own rate, then the effective rate is $\text{tax} \div \text{income}$.

```
income = 55_000

if income <= 10_000:
    tax = 0
elif income <= 40_000:
    tax = (income - 10_000) * 0.10
else:
    tax = (40_000 - 10_000) * 0.10 + (income - 40_000) * 0.20

effective_rate = tax / income
print(f"Income:           {income:>10,.2f}")
print(f"Tax due:          {tax:>10,.2f}")
print(f"Effective rate: {effective_rate:>10.2%}")
```

OUTPUT

```
Income:           55,000.00
Tax due:          6,000.00
Effective rate:    10.91%
```

CHAPTER 6

Repetition with Loops

Repetition: for loops over collections and ranges, accumulators, while loops, break/continue, validation loops, enumerate() and zip().

Libraries used: none: built-in Python only

Script 6.1 The for loop

ch06/s6_01_the_for_loop.py Built-in only · nothing to install

The loop variable takes each item of the list in turn, and the indented block runs once per item.

```
regions = ["North", "South", "East", "West"]

for region in regions:
    print(f"Preparing report for {region} region")
```

OUTPUT

```
Preparing report for North region
Preparing report for South region
Preparing report for East region
Preparing report for West region
```

Script 6.2 Looping over a string

ch06/s6_02_looping_over_a_string.py Built-in only · nothing to install

A string is a sequence too, so for visits it one character at a time.

```
for ch in "SKU":
    print(ch)
```

OUTPUT

```
S
K
U
```

Script 6.3 The range() function

ch06/s6_03_the_range_function.py

Built-in only · nothing to install

`range(1, 13)` produces 1 to 12 (the end is excluded). `:02d` pads the month with a leading zero.

```
for month in range(1, 13):  
    print(f"Closing books for month {month:02d}")
```

OUTPUT

```
Closing books for month 01  
Closing books for month 02  
Closing books for month 03  
Closing books for month 04  
Closing books for month 05  
Closing books for month 06  
Closing books for month 07  
Closing books for month 08  
Closing books for month 09  
Closing books for month 10  
Closing books for month 11  
Closing books for month 12
```

Script 6.4 Accumulators and counters

ch06/s6_04_accumulators_and_counters.py

Built-in only · nothing to install

`total += sale` builds a running sum; `high_days += 1` counts days that meet a condition.

```
daily_sales = [1250.00, 980.50, 1430.25, 1105.75, 1720.00]  
  
total = 0  
high_days = 0  
  
for sale in daily_sales:  
    total += sale  
    if sale > 1200:  
        high_days += 1  
  
average = total / len(daily_sales)  
print(f"Weekly total: {total:,.2f}")  
print(f"Daily average: {average:,.2f}")  
print(f"Days above 1,200: {high_days}")
```

OUTPUT

```
Weekly total: 6,486.50  
Daily average: 1,297.30  
Days above 1,200: 3
```

Script 6.5 Business example: loan amortization tablech06/s6_05_loan_amortization_table.py **Built-in only · nothing to install**

Each year: interest = balance × rate, principal = payment – interest, and the balance falls by the principal.

```
balance = 20_000
rate = 0.08
payment = 5_009.13      # annual payment for a 5-year loan

print(f"{'Year':<6}{ 'Interest':>12}{ 'Principal':>12}{ 'Balance':>12}")
for year in range(1, 6):
    interest = balance * rate
    principal = payment - interest
    balance -= principal
    print(f"{'year':<6}{interest:>12,.2f}{principal:>12,.2f}{balance:>12,.2f}")
```

OUTPUT

Year	Interest	Principal	Balance
1	1,600.00	3,409.13	16,590.87
2	1,327.27	3,681.86	12,909.01
3	1,032.72	3,976.41	8,932.60
4	714.61	4,294.52	4,638.08
5	371.05	4,638.08	-0.01

Script 6.6 The while loop: doubling savingsch06/s6_06_the_while_loop_doubling_savings.py **Built-in only · nothing to install**

while repeats as long as its condition is True; here, until savings reach the target.

```
savings = 1_000
target = 2_000
rate = 0.07
years = 0

while savings < target:
    savings *= 1 + rate
    years += 1

print(f"It takes {years} years to double the savings.")
print(f"Final balance: {savings:,.2f}")
```

OUTPUT

```
It takes 11 years to double the savings.
Final balance: 2,104.85
```

Script 6.7 break and continue

ch06/s6_07_break_and_continue.py

Built-in only · nothing to install

continue skips to the next item (refunds/zero rows); break stops the loop entirely (suspicious amount).

```
transactions = [250, -40, 1200, 0, 75000, 310]

for amount in transactions:
    if amount <= 0:
        continue          # ignore refunds and zero rows
    if amount > 50_000:
        print(f"Suspicious transaction of {amount:,} found. "
              "Stopping for review.")
        break
    print(f"Processed {amount}")
```

OUTPUT

```
Processed 250
Processed 1200
Suspicious transaction of 75,000 found. Stopping for review.
```

Script 6.8 Input validation loop

ch06/s6_08_input_validation_loop.py

Built-in only · nothing to install

while True keeps asking until the entry is valid, then break exits the loop.

```
while True:
    qty_text = input("Enter quantity (1-100): ")
    if qty_text.isdigit() and 1 <= int(qty_text) <= 100:
        quantity = int(qty_text)
        break
    print("Invalid entry, please try again.")

print(f"Quantity accepted: {quantity}")
```

SAMPLE SESSION · HIGHLIGHTED = TYPED BY THE USER

```
Enter quantity (1-100): abc
Invalid entry, please try again.
Enter quantity (1-100): 250
Invalid entry, please try again.
Enter quantity (1-100): 40
Quantity accepted: 40
```

Script 6.9 Nested loops

ch06/s6_09_nested_loops.py Built-in only · nothing to install

The inner loop runs completely for every pass of the outer loop: every store × every quarter.

```
stores = ["Hamra", "Jounieh"]
quarters = ["Q1", "Q2", "Q3"]

for store in stores:
    for q in quarters:
        print(f"{store} - {q}")
```

OUTPUT

```
Hamra - Q1
Hamra - Q2
Hamra - Q3
Jounieh - Q1
Jounieh - Q2
Jounieh - Q3
```

Script 6.10 enumerate() and zip()

ch06/s6_10_enumerate_and_zip.py Built-in only · nothing to install

enumerate() numbers the items; zip() walks two lists side by side.

```
products = ["Laptop", "Monitor", "Keyboard"]
prices = [950, 210, 45]

for i, product in enumerate(products, start=1):
    print(i, product)

for product, price in zip(products, prices):
    print(f"{product}: ${price}")
```

OUTPUT

```
1 Laptop
2 Monitor
3 Keyboard
Laptop: $950
Monitor: $210
Keyboard: $45
```

CHAPTER 7

Lists and Tuples

Ordered collections: creating and changing lists, list methods and built-ins, comprehensions, tables as lists of lists, and tuples.

Libraries used: none: built-in Python only

Script 7.1 Creating and changing a list

ch07/s7_01_creating_and_changing_a_list.py **Built-in only · nothing to install**

Lists hold ordered items in `[]`. Access by index, count with `len()`, and replace an item by assigning to its index.

```
monthly_sales = [42000, 38500, 45100, 51200]
employees = ["Nour", "Karim", "Hala", "Omar"]
mixed = ["INV-01", 250.0, True]
```

```
print(monthly_sales)
print(employees[0])
print(employees[-1])
print(len(employees))
```

```
employees[1] = "Karim A."
print(employees)
```

OUTPUT

```
[42000, 38500, 45100, 51200]
Nour
Omar
4
['Nour', 'Karim A.', 'Hala', 'Omar']
```

Script 7.2 List methods

ch07/s7_02_list_methods.py **Built-in only · nothing to install**

`append()` adds at the end, `insert()` at a position, `remove()` deletes by value, `pop()` removes and returns the last item.

```
cart = ["Notebook", "Pen"]
cart.append("Stapler")
cart.insert(0, "Backpack")
cart.remove("Pen")
last = cart.pop()

print(cart)
print("Removed:", last)
```

OUTPUT

```
['Backpack', 'Notebook']
Removed: Stapler
```

Script 7.3 Built-in functions for lists

ch07/s7_03_built_in_functions_for_lists.py **Built-in only · nothing to install**

`sum()`, `max()`, `min()`, `len()` and `sorted()` summarise a list in one line; `[:3]` takes the top three.

```
sales = [42000, 38500, 45100, 51200, 47300, 39900]

print("Total:", sum(sales))
print("Highest:", max(sales))
print("Lowest:", min(sales))
print("Average:", round(sum(sales) / len(sales), 2))
print("Sorted:", sorted(sales))
print("Top 3:", sorted(sales, reverse=True)[:3])
```

OUTPUT

```
Total: 264000
Highest: 51200
Lowest: 38500
Average: 44000.0
Sorted: [38500, 39900, 42000, 45100, 47300, 51200]
Top 3: [51200, 47300, 45100]
```

Script 7.4 Membership testing

ch07/s7_04_membership_testing.py

Built-in only · nothing to install

in checks whether a value is in the list, e.g. an approved-supplier check.

```
approved_suppliers = ["Nova Supplies", "Delta Parts", "Orion Tech"]
supplier = "Delta Parts"

if supplier in approved_suppliers:
    print("Supplier approved.")
```

OUTPUT

```
Supplier approved.
```

Script 7.5 List comprehensions

ch07/s7_05_list_comprehensions.py

Built-in only · nothing to install

[expression for item in list if condition] builds a new list in one line. The loop at the end is the long-hand equivalent.

```
prices = [120, 45, 300, 80, 15]

with_vat = [round(p * 1.11, 2) for p in prices]
expensive = [p for p in prices if p >= 100]

print(with_vat)
print(expensive)

# The same result written as an ordinary loop:
with_vat = []
for p in prices:
    with_vat.append(round(p * 1.11, 2))
```

OUTPUT

```
[133.2, 49.95, 333.0, 88.8, 16.65]
[120, 300]
```

Script 7.6 Lists of lists: order lines

ch07/s7_06_lists_of_lists_order_lines.py Built-in only · nothing to install

Each inner list is one row; unpacking in the for line names each column.

```
orders = [
    ["ORD-1", "Laptop", 2, 950],
    ["ORD-2", "Mouse", 10, 18],
    ["ORD-3", "Monitor", 3, 210],
]

grand_total = 0
for order_id, item, qty, price in orders:
    line_total = qty * price
    grand_total += line_total
    print(f"{order_id} {item:<8} {qty:>3} x {price:>6} = {line_total:>7,}")

print(f"Grand total: {grand_total:,}")
```

OUTPUT

```
ORD-1 Laptop      2 x    950 =   1,900
ORD-2 Mouse     10 x     18 =     180
ORD-3 Monitor     3 x    210 =     630
Grand total: 2,710
```

Script 7.7 Copying lists

ch07/s7_07_copying_lists.py Built-in only · nothing to install

alias = original points to the same list; .copy() makes an independent copy.

```
original = [1, 2, 3]
alias = original
copy = original.copy()

alias.append(4)
print(original)
print(copy)
```

OUTPUT

```
[1, 2, 3, 4]
[1, 2, 3]
```

Script 7.8 Tuples

ch07/s7_08_tuples.py Built-in only · nothing to install

Tuples use `()` and cannot be changed, making them ideal for fixed records. They can be unpacked into variables.

```
headquarters = (33.8938, 35.5018)           # latitude, longitude
fiscal_quarter = ("Q2", 2026)

print(headquarters[0])
lat, lon = headquarters
print(f"Latitude {lat}, longitude {lon}")
```

OUTPUT

```
33.8938
Latitude 33.8938, longitude 35.5018
```

Script 7.9 Business case: underperforming stores

ch07/s7_09_underperforming_stores.py Built-in only · nothing to install

Flags stores below 80% of the chain average using `zip()` inside a list comprehension.

```
stores = ["Hamra", "Achrafieh", "Jounieh", "Tripoli", "Saida", "Zahle"]
sales = [58000, 71000, 39000, 44500, 62000, 36000]

average = sum(sales) / len(sales)
threshold = 0.8 * average

print(f"Chain average: {average:,.0f}   Threshold: {threshold:,.0f}")
flagged = [s for s, v in zip(stores, sales) if v < threshold]
print("Stores needing review:", ", ".join(flagged))
```

OUTPUT

```
Chain average: 51,750   Threshold: 41,400
Stores needing review: Jounieh, Zahle
```

CHAPTER 8

Dictionaries and Sets

Key-value records with dictionaries, safe lookups, counting and grouping, nested records, and sets for unique values and comparisons.

Libraries used: none: built-in Python only

Script 8.1 Creating and editing a dictionary

ch08/s8_01_creating_and_editing_a_dictionary.py **Built-in only · nothing to install**

A dictionary maps keys to values in {}. Assign to add or change a key; del removes one.

```
product = {
    "sku": "LAP-0932",
    "name": "Business Laptop 14",
    "price": 949.00,
    "in_stock": 37,
}

print(product["name"])
print(product["price"])

product["price"] = 899.00          # change a value
product["category"] = "Computers" # add a new key
del product["in_stock"]          # remove a key

print(product)
```

OUTPUT

```
Business Laptop 14
949.0
{'sku': 'LAP-0932', 'name': 'Business Laptop 14', 'price': 899.0, 'category':
'Computers'}
```

Script 8.2 Safe lookups with get()

ch08/s8_02_safe_lookups_with_get.py **Built-in only · nothing to install**

get() returns None, or your chosen default, instead of crashing when a key is missing.

```
exchange_rates = {"EUR": 0.92, "GBP": 0.79, "JPY": 151.3}

print(exchange_rates.get("EUR"))
print(exchange_rates.get("CHF", "rate not available"))
```

OUTPUT

```
0.92
rate not available
```

Script 8.3 Looping through a dictionary

ch08/s8_03_looping_through_a_dictionary.py **Built-in only · nothing to install**

.items() gives key and value pairs; .values() gives just the values, e.g. for a total.

```
department_budget = {
    "Marketing": 120_000,
    "IT": 250_000,
    "HR": 80_000,
    "Operations": 310_000,
}

for dept, budget in department_budget.items():
    print(f"{dept:<12}{budget:>10,}")

total = sum(department_budget.values())
print(f"{'Total':<12}{total:>10,}")
```

OUTPUT

```
Marketing      120,000
IT             250,000
HR             80,000
Operations     310,000
Total         760,000
```

Script 8.4 Counting with a dictionary

ch08/s8_04_counting_with_a_dictionary.py **Built-in only · nothing to install**

counts.get(item, 0) + 1 is the standard counting pattern; max(..., key=counts.get) finds the best seller.

```
orders = ["Laptop", "Mouse", "Laptop", "Monitor", "Mouse", "Laptop"]

counts = {}
for item in orders:
    counts[item] = counts.get(item, 0) + 1

print(counts)
best_seller = max(counts, key=counts.get)
print("Best seller:", best_seller)
```

OUTPUT

```
{'Laptop': 3, 'Mouse': 2, 'Monitor': 1}
Best seller: Laptop
```

Script 8.5 Nested dictionaries: customer records

ch08/s8_05_nested_dictionaries_customer_records.py **Built-in only · nothing to install**

A dictionary of dictionaries: look up a customer by ID, then a field by name.

```
customers = {
    "C001": {"name": "Rami Khoury", "city": "Beirut", "balance": 1200.0},
    "C002": {"name": "Lea Saab", "city": "Byblos", "balance": 0.0},
    "C003": {"name": "Ziad Harb", "city": "Beirut", "balance": 450.5},
}

print(customers["C003"]["name"])

for cid, info in customers.items():
    if info["balance"] > 0:
        print(f"{cid} {info['name']} owes {info['balance']:,.2f}")
```

OUTPUT

```
Ziad Harb
C001 Rami Khoury owes 1,200.00
C003 Ziad Harb owes 450.50
```

Script 8.6 Grouping a list of records

ch08/s8_06_grouping_a_list_of_records.py **Built-in only · nothing to install**

Totals sales by region by accumulating into a dictionary, like a mini pivot table.

```
sales = [
    {"region": "North", "rep": "Hala", "amount": 12500},
    {"region": "South", "rep": "Omar", "amount": 9800},
    {"region": "North", "rep": "Karim", "amount": 15200},
]

by_region = {}
for row in sales:
    by_region[row["region"]] = (by_region.get(row["region"], 0)
                               + row["amount"])

print(by_region)
```

OUTPUT

```
{'North': 27700, 'South': 9800}
```

Script 8.7 Dictionary comprehensions

ch08/s8_07_dictionary_comprehensions.py **Built-in only · nothing to install**

{key: value for ...} builds a new dictionary in one line, here converting prices to EUR.

```
prices_usd = {"Laptop": 950, "Mouse": 18, "Monitor": 210}
prices_eur = {item: round(p * 0.92, 2) for item, p in prices_usd.items()}
print(prices_eur)
```

OUTPUT

```
{'Laptop': 874.0, 'Mouse': 16.56, 'Monitor': 193.2}
```

Script 8.8 Sets: unique values

ch08/s8_08_sets_unique_values.py **Built-in only · nothing to install**

set() removes duplicates; len() then counts unique customers. Sets are unordered.

```
purchase_log = ["C001", "C002", "C001", "C004", "C002", "C005"]
unique_customers = set(purchase_log)
print(unique_customers)
print("Number of unique customers:", len(unique_customers))
```

OUTPUT

```
{'C001', 'C002', 'C004', 'C005'}
Number of unique customers: 4
```

Script 8.9 Set operations for business questions

ch08/s8_09_set_operations_for_business_questions.py **Built-in only · nothing to install**

& = in both, | = in either, - = in the first but not the second.

```
newsletter = {"C001", "C002", "C003", "C006"}
purchased = {"C002", "C003", "C004", "C005"}

print("Both:", newsletter & purchased)
print("Either:", newsletter | purchased)
print("Subscribed but never bought:", newsletter - purchased)
print("Bought but not subscribed:", purchased - newsletter)
```

OUTPUT

```
Both: {'C002', 'C003'}
Either: {'C003', 'C005', 'C001', 'C002', 'C004', 'C006'}
Subscribed but never bought: {'C001', 'C006'}
Bought but not subscribed: {'C004', 'C005'}
```

CHAPTER 9

Functions

Packaging logic into reusable, named, documented functions: parameters, defaults, return values, keyword arguments, scope, lambdas and type hints.

Libraries used: none: built-in Python only

Script 9.1 Defining and calling a function

ch09/s9_01_defining_and_calling_a_function.py **Built-in only · nothing to install**

`def` creates a function; `return` sends a result back. Note the tiny float error in 111.00000000000001 (see Script 2.9).

```
def add_vat(amount):  
    return amount * 1.11  
  
print(add_vat(100))  
print(add_vat(250))
```

OUTPUT

```
111.00000000000001  
277.5
```

Script 9.2 Parameters, defaults and docstrings

ch09/s9_02_parameters_defaults_and_docstrings.py **Built-in only · nothing to install**

`discount=0.0` is a default value. The triple-quoted docstring documents the function, and `help()` displays it.

```
def line_total(quantity, unit_price, discount=0.0):  
    """Return the total for an order line after discount."""  
    gross = quantity * unit_price  
    return round(gross * (1 - discount), 2)  
  
print(line_total(4, 25))  
print(line_total(4, 25, 0.10))  
  
help(line_total)
```

OUTPUT

```
100.0
90.0
Help on function line_total in module __main__:

line_total(quantity, unit_price, discount=0.0)
    Return the total for an order line after discount.
```

Script 9.3 Returning several values

ch09/s9_03_returning_several_values.py **Built-in only · nothing to install**

Returning a, b, c sends back a tuple that can be unpacked into separate variables.

```
def summarize(values):
    return min(values), max(values), sum(values) / len(values)

low, high, avg = summarize([120, 340, 95, 410])
print(f"Low {low}, high {high}, average {avg:.2f}")
```

OUTPUT

```
Low 95, high 410, average 241.25
```

Script 9.4 Keyword arguments: loan payment

ch09/s9_04_keyword_arguments_loan_payment.py **Built-in only · nothing to install**

Naming arguments (principal=..., years=...) makes calls readable and order-independent.

```
def loan_payment(principal, annual_rate, years):
    """Return the fixed annual payment for an amortizing loan."""
    r = annual_rate
    return principal * r / (1 - (1 + r) ** -years)

pay = loan_payment(principal=20_000, years=5, annual_rate=0.08)
print(f"Annual payment: {pay:,.2f}")
```

OUTPUT

```
Annual payment: 5,009.13
```

Script 9.5 Functions without return

ch09/s9_05_functions_without_return.py **Built-in only · nothing to install**

A function can simply do something, such as print a report header. It then returns `None`.

```
def print_header(title):
    print("=" * 40)
    print(f"{{title:^40}}")
    print("=" * 40)

print_header("MONTHLY SALES REPORT")
```

OUTPUT

```
=====
MONTHLY SALES REPORT
=====
```

Script 9.6 Scope: global and local variables

ch09/s9_06_scope_global_and_local_variables.py **Built-in only · nothing to install**

Variables created inside a function are local and disappear afterwards; constants defined at the top are readable everywhere.

```
VAT_RATE = 0.11                                # global constant

def price_with_vat(net):
    vat = net * VAT_RATE # vat is local
    return net + vat

print(price_with_vat(200))
# print(vat) would raise NameError: name 'vat' is not defined
```

OUTPUT

```
222.0
```

Script 9.7 Functions calling functions

ch09/s9_07_functions_calling_functions.py **Built-in only · nothing to install**

Small single-purpose functions combine into a complete invoice calculation.

```
def subtotal(items):
    return sum(q * p for q, p in items)

def apply_discount(amount, rate):
    return amount * (1 - rate)

def add_tax(amount, rate=0.11):
    return amount * (1 + rate)

def invoice_total(items, discount_rate):
    return round(add_tax(apply_discount(subtotal(items), discount_rate)), 2)

cart = [(2, 950), (10, 18), (3, 210)]
print(invoice_total(cart, 0.05))
```

OUTPUT

```
2857.7
```

Script 9.8 Lambda functions

ch09/s9_08_lambda_functions.py **Built-in only · nothing to install**

lambda creates a tiny unnamed function, most often used as a sort key.

```
employees = [("Hala", 3200), ("Omar", 2750), ("Nour", 4100)]

by_salary = sorted(employees, key=lambda e: e[1], reverse=True)
print(by_salary)
```

OUTPUT

```
[('Nour', 4100), ('Hala', 3200), ('Omar', 2750)]
```

Script 9.9 Type hints

ch09/s9_09_type_hints.py **Built-in only · nothing to install**

Annotations such as revenue: float and -> float document expected types. Python does not enforce them.

```
def net_profit(revenue: float, costs: float) -> float:
    return revenue - costs

print(net_profit(50_000, 32_500))
```

OUTPUT

17500

One demonstration line was added at the end so the script prints a visible result.

Script 9.10 Business case: a pricing toolkitch09/s9_10_a_pricing_toolkit.py **Built-in only · nothing to install**

Markup pricing, .99 psychological pricing and gross margin as three small functions.

```
def selling_price(cost: float, markup: float = 0.40) -> float:
    """Price from cost using a percentage markup."""
    return round(cost * (1 + markup), 2)

def gross_margin(price: float, cost: float) -> float:
    """Margin as a fraction of the selling price."""
    return (price - cost) / price

def psychological_price(price: float) -> float:
    """Round up to the next .99 ending."""
    return int(price) + 0.99

cost = 42.30
price = selling_price(cost)
nice_price = psychological_price(price)

print(f"Cost:                {cost:.2f}")
print(f"Markup price:         {price:.2f}")
print(f"Display price:          {nice_price:.2f}")
print(f"Gross margin:           {gross_margin(nice_price, cost):.1%}")
```

OUTPUT

```
Cost:                42.30
Markup price:         59.22
Display price:        59.99
Gross margin:         29.5%
```

CHAPTER 10

Modules and the Standard Library

Importing ready-made tools: math, statistics, random, datetime, writing your own module, and installing third-party packages.

Libraries used: datetime, finance_tools, math, random, statistics

Script 10.1 Three ways to import

ch10/s10_01_three_ways_to_import.py [Standard library · math](#) [Standard library · statistics](#)

import module, from module import name, and import module as alias.

```
import math
print(math.sqrt(144))

from math import ceil, floor
print(ceil(4.2), floor(4.8))

import statistics as st
print(st.mean([10, 20, 30]))
```

OUTPUT

```
12.0
5 4
20
```

Script 10.2 The math module

ch10/s10_02_the_math_module.py [Standard library · math](#)

math.ceil() always rounds up (you can't ship part of a box); math.exp() is used for continuous compounding.

```
import math

boxes_needed = math.ceil(1_030 / 48) # always round up for packaging
print("Boxes needed:", boxes_needed)

# continuous compounding: A = P * e^(rt)
amount = 5_000 * math.exp(0.05 * 3)
print(f"Continuously compounded: {amount:,.2f}")
```

OUTPUT

```
Boxes needed: 22
Continuously compounded: 5,809.17
```

Script 10.3 The statistics module

ch10/s10_03_the_statistics_module.py [Standard library · statistics](#)

mean, median and standard deviation of service response times.

```
import statistics as st

response_times = [2.1, 3.4, 1.8, 2.9, 5.6, 2.2, 3.1, 2.4]

print("Mean:", round(st.mean(response_times), 2))
print("Median:", st.median(response_times))
print("Std dev:", round(st.stdev(response_times), 2))
```

OUTPUT

```
Mean: 2.94
Median: 2.65
Std dev: 1.2
```

Script 10.4 The random module

ch10/s10_04_the_random_module.py [Standard library · random](#)

seed() makes random results reproducible; randint, choice and sample pick random values.

```
import random

random.seed(42)    # makes results reproducible
print(random.randint(1, 100))
print(random.choice(["Hala", "Omar", "Nour"]))
print(random.sample(range(1, 50), 5))
```

OUTPUT

```
82
Hala
[2, 48, 18, 16, 15]
```

Random numbers can differ between Python versions even with the same seed. What matters is that your results repeat each time you run.

Script 10.5 Monte Carlo profit simulationch10/s10_05_monte_carlo_profit_simulation.py [Standard library · random](#)

Runs 10,000 random demand and cost scenarios to estimate the probability of making a profit.

```
import random
random.seed(1)

simulations = 10_000
profitable = 0

for _ in range(simulations):
    demand = random.randint(800, 1500)      # uncertain demand
    unit_cost = random.uniform(9, 13)        # uncertain cost
    profit = demand * (18 - unit_cost) - 6_000
    if profit > 0:
        profitable += 1

print(f"Probability of profit: {profitable / simulations:.1%}")
```

OUTPUT

Probability of profit: 84.8%

Random numbers can differ between Python versions even with the same seed. What matters is that your results repeat each time you run.

Script 10.6 Working with dates: datetimech10/s10_06_working_with_dates_datetime.py [Standard library · datetime](#)

timedelta adds days to a date; subtracting dates gives the days between; strftime()/strptime() format and parse dates.

```
from datetime import date, datetime, timedelta

today = date(2026, 9, 22)
invoice_date = date(2026, 8, 15)
due_date = invoice_date + timedelta(days=30)

print("Due date:", due_date)
print("Days overdue:", (today - due_date).days)
print("Formatted:", invoice_date.strftime("%d %B %Y"))

text = "03/15/2026"
d = datetime.strptime(text, "%m/%d/%Y").date()
print(d, d.strftime("%A"))
```

OUTPUT

```
Due date: 2026-09-14
Days overdue: 8
Formatted: 15 August 2026
2026-03-15 Sunday
```

Script 10.7 Writing your own module

ch10/finance_tools.py + main.py [Standard library · finance_tools \(your own module\)](#)

Save the first part as `finance_tools.py`; any script in the same folder can then import it.

```
FINANCE_TOOLS.PY
"""Reusable financial helper functions."""

def future_value(p, r, n):
    return p * (1 + r) ** n

def present_value(fv, r, n):
    return fv / (1 + r) ** n
```

MAIN.PY (SAME FOLDER)

```
import finance_tools as ft
print(round(ft.present_value(10_000, 0.08, 5), 2))
```

OUTPUT

```
6805.83
```

CHAPTER 11

Working with Files

Saving and loading data: text files, an append-only log, CSV reading and writing, JSON configuration, and folders with `pathlib`.

Libraries used: `csv`, `datetime`, `json`, `pathlib`

Script 11.1 Writing and reading a text file

`ch11/s11_01_writing_and_reading_a_text_file.py` Built-in only · nothing to install

"w" writes (replacing any old content); with closes the file automatically. Reading line by line works even for huge files.

```
with open("notes.txt", "w") as f:
    f.write("Meeting with supplier on Monday\n")
    f.write("Review Q3 budget\n")

with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

with open("notes.txt") as f:
    for line_number, line in enumerate(f, start=1):
        print(line_number, line.strip())
```

OUTPUT

```
Meeting with supplier on Monday
Review Q3 budget
```

```
1 Meeting with supplier on Monday
2 Review Q3 budget
```

Script 11.2 Appending: an activity log

ch11/s11_02_appending_an_activity_log.py [Standard library · datetime](#)

"a" appends to the end of the file, so each call adds a time-stamped line.

```
from datetime import datetime

def log(message):
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("activity.log", "a") as f:
        f.write(f"{stamp} {message}\n")

log("User admin logged in")
log("Report generated: sales_q3.csv")
```

NO SCREEN OUTPUT · CREATES ACTIVITY.LOG, WHICH CONTAINS:

```
2026-09-24 19:12:45 User admin logged in
2026-09-24 19:12:45 Report generated: sales_q3.csv
```

Your timestamps will show the moment you run it. Run it twice and the log grows, because "a" appends.

Script 11.3 Reading a CSV file

ch11/s11_03_reading_a_csv_file.py [Standard library · csv](#)

csv.DictReader turns each row into a dictionary keyed by the header names. Needs sales.csv (included in the downloads).

Data file needed in the same folder: [sales.csv](#) (included in the download).

```
import csv

revenue_by_region = {}

with open("sales.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        revenue = int(row["units"]) * float(row["unit_price"])
        region = row["region"]
        revenue_by_region[region] = (revenue_by_region.get(region, 0)
                                     + revenue)

for region, revenue in sorted(revenue_by_region.items()):
    print(f"{region:<8}{revenue:>10,.2f}")
```

OUTPUT

```
East      1,900.00
North     3,690.00
South     1,400.00
```

Script 11.4 Writing a CSV filech11/s11_04_writing_a_csv_file.py Standard library · csv

csv.DictWriter writes a header row and then one row per dictionary. `newline=""` prevents blank lines on Windows.

```
import csv

summary = [
    {"region": "East", "revenue": 1900.0},
    {"region": "North", "revenue": 3690.0},
    {"region": "South", "revenue": 1400.0},
]

with open("region_summary.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["region", "revenue"])
    writer.writeheader()
    writer.writerows(summary)
```

NO SCREEN OUTPUT · CREATES REGION_SUMMARY.CSV, WHICH CONTAINS:

```
region,revenue
East,1900.0
North,3690.0
South,1400.0
```

Script 11.5 JSON for settings and recordsch11/s11_05_json_for_settings_and_records.py Standard library · json

json.dump() saves Python dictionaries and lists as text; json.load() reads them back.

```
import json

config = {
    "company": "Cedar Trading Co.",
    "vat_rate": 0.11,
    "currencies": ["USD", "LBP", "EUR"],
    "reorder_level": 10,
}

with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

with open("config.json") as f:
    loaded = json.load(f)

print(loaded["vat_rate"], loaded["currencies"])
```

OUTPUT

```
0.11 ['USD', 'LBP', 'EUR']
```

NO SCREEN OUTPUT · CREATES CONFIG.JSON, WHICH CONTAINS:

```
{
  "company": "Cedar Trading Co.",
  "vat_rate": 0.11,
  "currencies": [
    "USD",
    "LBP",
    "EUR"
  ],
  "reorder_level": 10
}
```

Script 11.6 Paths and folders with pathlib

ch11/s11_06_paths_and_folders_with_pathlib.py [Standard library · pathlib](#)

Path objects create folders (mkdir) and find files by pattern (glob).

```
from pathlib import Path

reports = Path("reports")
reports.mkdir(exist_ok=True)

for csv_file in Path(".").glob("*.csv"):
    print(csv_file.name, csv_file.stat().st_size, "bytes")
```

OUTPUT

```
sales.csv 186 bytes
```

The list you see depends on which CSV files are in your current folder.

Script 11.7 Business case: merging branch exports

ch11/s11_07_merging_branch_exports.py [Standard library · csv](#) [Standard library · pathlib](#)

Combines every exports/branch_*.csv into one total per product, sorted from highest to lowest.

Data file needed in the same folder: exports/branch_hamra.csv, exports/branch_jounieh.csv, exports/branch_tripoli.csv (included in the download).

```
import csv
from pathlib import Path

totals = {}
for path in Path("exports").glob("branch_*.csv"):
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            totals[row["product"]] = (totals.get(row["product"], 0)
                                     + int(row["units"]))

with open("combined_totals.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["product", "total_units"])
    for product, units in sorted(totals.items(), key=lambda x: - x[1]):
        writer.writerow([product, units])

print(f"Combined {len(totals)} products into combined_totals.csv")
```

OUTPUT

Combined 4 products into combined_totals.csv

NO SCREEN OUTPUT · CREATES COMBINED_TOTALS.CSV, WHICH CONTAINS:

```
product,total_units
Mouse,73
Keyboard,21
Laptop,11
Monitor,9
```

CHAPTER 12

Errors, Exceptions and Debugging

Reading tracebacks, handling exceptions with try/except/else/finally, raising your own errors, and defensive coding with assert.

Libraries used: none: built-in Python only

Script 12.1 Reading a traceback

ch12/s12_01_reading_a_traceback.py **Built-in only · nothing to install**

Dividing by zero crashes the program. Read the traceback from the bottom up: the error type, then the line that caused it.

```
def average_order(total, count):
    return total / count

print(average_order(5000, 0))
```

ERROR MESSAGE (EXPECTED)

```
Traceback (most recent call last):
  File "s12_01_reading_a_traceback.py", line 4, in <module>
    print(average_order(5000, 0))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "s12_01_reading_a_traceback.py", line 2, in average_order
    return total / count
    ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

Script 12.2 try and except

ch12/s12_02_try_and_except.py **Built-in only · nothing to install**

If int() fails, Python jumps to the except block instead of crashing.

```
try:
    quantity = int(input("Quantity: "))
    print("Accepted:", quantity)
except ValueError:
    print("Please enter a whole number.")
```

SAMPLE SESSION · **HIGHLIGHTED** = TYPED BY THE USER

Quantity: **ten**
Please enter a whole number.

Script 12.3 Handling several exception typesch12/s12_03_handling_several_exception_types.py **Built-in only · nothing to install**

Separate except blocks handle different problems; as e gives access to the error message.

```
def safe_ratio(numerator, denominator):
    try:
        return numerator / denominator
    except ZeroDivisionError:
        return None
    except TypeError as e:
        print("Invalid input:", e)
        return None

print(safe_ratio(450, 9))
print(safe_ratio(450, 0))
print(safe_ratio("450", 9))
```

OUTPUT

```
50.0
None
Invalid input: unsupported operand type(s) for /: 'str' and 'int'
None
```

Script 12.4 else and finallych12/s12_04_else_and_finally.py **Built-in only · nothing to install**

else runs only if no error occurred; finally always runs, which is ideal for clean-up messages.

```
try:
    f = open("sales.csv")
except FileNotFoundError:
    print("sales.csv not found. Check the export folder.")
else:
    print("File opened, rows:", len(f.readlines()) - 1)
    f.close()
finally:
    print("Import step finished.")
```

OUTPUT

```
File opened, rows: 5
Import step finished.
```

Script 12.5 A robust input functionch12/s12_05_a_robust_input_function.py **Built-in only · nothing to install**

Combines a loop with try/except so the user is asked again until a valid number is entered.

```
def ask_float(prompt, minimum=0.0):
    while True:
        try:
            value = float(input(prompt))
            if value < minimum:
                print(f"Value must be at least {minimum}.")
                continue
            return value
        except ValueError:
            print("That is not a number. Try again.")

price = ask_float("Unit price: ")
print("Price accepted:", price)
```

SAMPLE SESSION · HIGHLIGHTED = TYPED BY THE USER

```
Unit price: abc
That is not a number. Try again.
Unit price: -5
Value must be at least 0.0.
Unit price: 19.99
Price accepted: 19.99
```

One demonstration line was added at the end so the script prints a visible result.

Script 12.6 Raising exceptionsch12/s12_06_raising_exceptions.py **Built-in only · nothing to install**

raise ValueError(...) enforces business rules such as no overdrafts; the caller decides how to respond.

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Withdrawal amount must be positive.")
    if amount > balance:
        raise ValueError("Insufficient funds.")
    return balance - amount

try:
    new_balance = withdraw(500, 800)
except ValueError as e:
    print("Transaction rejected:", e)
```

OUTPUT

Transaction rejected: Insufficient funds.

Script 12.7 Processing data without stopping

ch12/s12_07_processing_data_without_stopping.py **Built-in only · nothing to install**

Bad rows are recorded and skipped, so one faulty value does not stop the whole import.

```
rows = ["120.50", "98", "N/A", "300.00", "", "45.25"]

valid, errors = [], []
for i, text in enumerate(rows, start=1):
    try:
        valid.append(float(text))
    except ValueError:
        errors.append(i)

print("Total of valid rows:", sum(valid))
print("Rows with problems:", errors)
```

OUTPUT

Total of valid rows: 563.75
Rows with problems: [3, 5]

Script 12.8 Debugging with assert

ch12/s12_08_debugging_with_assert.py **Built-in only · nothing to install**

assert stops the program with a message if an assumption is false, catching bad inputs early.

```
def apply_discount(price, rate):
    assert 0 <= rate <= 1, "rate must be between 0 and 1"
    return price * (1 - rate)

print(apply_discount(200, 0.25))
```

OUTPUT

150.0

CHAPTER 13

Object-Oriented Programming Basics

Classes bundle data (attributes) and behaviour (methods): products, customers, bank accounts and an employee hierarchy.

Libraries used: none: built-in Python only

Script 13.1 Defining a class: Product

ch13/s13_01_defining_a_class_product.py Built-in only · nothing to install

`__init__` sets up each new object; `self` refers to that object. Methods such as `restock()` change its state.

```
class Product:
    def __init__(self, sku, name, price, stock=0):
        self.sku = sku
        self.name = name
        self.price = price
        self.stock = stock

    def stock_value(self):
        return self.price * self.stock

    def restock(self, units):
        self.stock += units

laptop = Product("LAP-0932", "Business Laptop 14", 949.0, 12)
mouse = Product("MOU-0101", "Wireless Mouse", 18.5)

laptop.restock(8)
print(laptop.name, laptop.stock)
print(f"Stock value: {laptop.stock_value():.2f}")
print(mouse.stock)
```

OUTPUT

```
Business Laptop 14 20
Stock value: 18,980.00
0
```

Script 13.2 A readable representation with `__str__`ch13/s13_02_a_readable_representation_with_str.py Built-in only · nothing to install`__str__` controls what `print(object)` shows.

```
class Customer:
    def __init__(self, customer_id, name, email):
        self.customer_id = customer_id
        self.name = name
        self.email = email.lower().strip()

    def __str__(self):
        return f"{self.customer_id}: {self.name} <{self.email}>"

c = Customer("C001", "Rami Khoury", " Rami.K@Example.com ")
print(c)
```

OUTPUT

```
C001: Rami Khoury <rami.k@example.com>
```

Script 13.3 Encapsulation: BankAccountch13/s13_03_encapsulation_bankaccount.py **Built-in only · nothing to install**

Underscore attributes signal internal data; @property exposes the balance read-only, and methods enforce the rules.

```
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self._balance = balance
        self._transactions = []

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive.")
        self._balance += amount
        self._transactions.append(("deposit", amount))

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("Insufficient funds.")
        self._balance -= amount
        self._transactions.append(("withdrawal", amount))

    @property
    def balance(self):
        return self._balance

    def statement(self):
        for kind, amount in self._transactions:
            print(f"{kind:<12}{amount:>10,.2f}")
        print(f"{'balance':<12}{self._balance:>10,.2f}")

acct = BankAccount("Lea Saab", 1000)
acct.deposit(250)
acct.withdraw(400)
acct.statement()
```

OUTPUT

deposit	250.00
withdrawal	400.00
balance	850.00

Script 13.4 Class attributesch13/s13_04_class_attributes.py **Built-in only · nothing to install**

company and raise_rate belong to the class and are shared by every employee object.

```
class Employee:
    company = "Cedar Trading Co."
    raise_rate = 0.05

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def apply_raise(self):
        self.salary = round(self.salary * (1 + Employee.raise_rate), 2)

e1 = Employee("Hala", 3200)
e2 = Employee("Omar", 2750)
e1.apply_raise()
print(e1.name, e1.salary, e1.company)
print(e2.name, e2.salary, e2.company)
```

OUTPUT

```
Hala 3360.0 Cedar Trading Co.
Omar 2750 Cedar Trading Co.
```

Script 13.5 Inheritancech13/s13_05_inheritance.py **Built-in only · nothing to install**

Manager and Contractor inherit from Employee and override `annual_cost()`; `super()` reuses the parent's code.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def annual_cost(self):
        return self.salary * 12

    def __str__(self):
        return f"{self.name} ({type(self).__name__})"

class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus

    def annual_cost(self):
        return super().annual_cost() + self.bonus

class Contractor(Employee):
    def annual_cost(self):
        return self.salary * 10           # paid for ten months only

staff = [
    Employee("Nour", 2500),
    Manager("Karim", 4200, 6000),
    Contractor("Ziad", 3000),
]

for person in staff:
    print(f"{str(person):<22}{person.annual_cost():>10,}")
```

OUTPUT

Nour (Employee)	30,000
Karim (Manager)	56,400
Ziad (Contractor)	30,000

CHAPTER 14

Introduction to Data Analysis with pandas

Tables (DataFrames) in code: creating, loading, inspecting, selecting, calculating, filtering, sorting, grouping, cleaning and exporting.

Libraries used: numpy, openpyxl, pandas

Script 14.1 Creating a DataFrame

ch14/s14_01_creating_a_dataframe.py [pip install pandas](#)

A dictionary of equal-length lists becomes a table; each key is a column.

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT

	order_id	region	product	units	unit_price
0	101	North	Laptop	3	950
1	102	South	Mouse	25	18
2	103	North	Monitor	4	210
3	104	East	Laptop	2	950
4	105	South	Laptop	1	950
5	106	East	Mouse	30	18

Script 14.2 Loading data from CSV and Excel

ch14/s14_02_loading_data_from_csv_and_excel.py [pip install openpyxl](#) [pip install pandas](#)

`read_csv()` and `read_excel()` load files straight into a DataFrame. Excel files require `openpyxl`.

Data file needed in the same folder: `sales.csv`, `sales.xlsx` (included in the download).

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd
```

```
df = pd.read_csv("sales.csv")
df = pd.read_excel("sales.xlsx", sheet_name="September")
```

OUTPUT

(nothing printed: the data is loaded into df. Add print(df.head()) to see it)

Script 14.3 Inspecting data

ch14/s14_03_inspecting_data.py [pip install pandas](#)

shape, head(), dtypes and describe() give a first look at any new dataset.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
```

```
print(df.shape)           # (rows, columns)
print(df.head(3))         # first three rows
print(df.dtypes)          # column types
print(df.describe())       # summary statistics for numeric columns
```

OUTPUT

```
(6, 5)
  order_id region product units unit_price
0      101  North  Laptop      3        950
1      102  South  Mouse     25         18
2      103  North  Monitor     4        210
order_id      int64
region         str
product        str
units          int64
unit_price     int64
dtype: object
```

	order_id	units	unit_price
count	6.000000	6.000000	6.000000
mean	103.500000	10.833333	516.000000
std	1.870829	13.044795	480.564668
min	101.000000	1.000000	18.000000
25%	102.250000	2.250000	66.000000
50%	103.500000	3.500000	580.000000
75%	104.750000	19.750000	950.000000
max	106.000000	30.000000	950.000000

Column types may print as `object` or `str` depending on your pandas version. Both mean text.

Script 14.4 Selecting columns and rows

ch14/s14_04_selecting_columns_and_rows.py [pip install pandas](#)

`df["col"]` returns one column; `df[...]` several; `loc` selects by label and `iloc` by position.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
```

```
print(df["product"])           # one column (a Series)
print(df[["region", "units"]]) # several columns (a DataFrame)
print(df.loc[2])              # row with index label 2
print(df.iloc[0:2])           # first two rows by position
```

OUTPUT

```

0    Laptop
1    Mouse
2    Monitor
3    Laptop
4    Laptop
5    Mouse
Name: product, dtype: str
   region  units
0  North     3
1  South    25
2  North     4
3  East      2
4  South     1
5  East    30
order_id    103
region      North
product      Monitor
units         4
unit_price   210
Name: 2, dtype: object
   order_id  region  product  units  unit_price
0         101  North  Laptop      3         950
1         102  South  Mouse    25          18

```

Column types may print as `object` or `str` depending on your pandas version. Both mean text.

Script 14.5 Calculated columns

ch14/s14_05_calculated_columns.py [pip install pandas](#)

Arithmetic on whole columns creates new columns, with no loop required.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```

import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)

```

```

df["revenue"] = df["units"] * df["unit_price"]
df["vat"] = df["revenue"] * 0.11
print(df[["order_id", "product", "revenue", "vat"]])

```

OUTPUT

	order_id	product	revenue	vat
0	101	Laptop	2850	313.5
1	102	Mouse	450	49.5
2	103	Monitor	840	92.4
3	104	Laptop	1900	209.0
4	105	Laptop	950	104.5
5	106	Mouse	540	59.4

Script 14.6 Filtering rowsch14/s14_06_filtering_rows.py [pip install pandas](#)

Put a condition inside `df[...]`. Combine conditions with `&` (and) and `|` (or), each wrapped in parentheses. `isin()` matches a list.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
df["revenue"] = df["units"] * df["unit_price"]
```

```
big_orders = df[df["revenue"] > 1000]
north_laptops = df[(df["region"] == "North") & (df["product"] == "Laptop")]
selected = df[df["region"].isin(["East", "South"])]

print(big_orders[["order_id", "revenue"]])
```

OUTPUT

	order_id	revenue
0	101	2850
3	104	1900

Script 14.7 **Sorting**ch14/s14_07_sorting.py [pip install pandas](#)

`sort_values()` orders rows; `ascending=False` puts the largest first, and `head(3)` keeps the top three.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
df["revenue"] = df["units"] * df["unit_price"]
```

```
top = df.sort_values("revenue", ascending=False).head(3)
print(top[["order_id", "product", "revenue"]])
```

OUTPUT

	order_id	product	revenue
0	101	Laptop	2850
3	104	Laptop	1900
4	105	Laptop	950

Script 14.8 **Grouping, aggregation and pivot tables**ch14/s14_08_grouping_aggregation_and_pivot_tables.py [pip install pandas](#)

`groupby()` + `sum()` totals by category; `agg()` computes several summaries; `pivot_table()` works like an Excel pivot.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
df["revenue"] = df["units"] * df["unit_price"]
```

```

by_region = df.groupby("region")["revenue"].sum()
print(by_region)

summary = df.groupby("product").agg(
    orders=("order_id", "count"),
    units=("units", "sum"),
    revenue=("revenue", "sum"),
)
print(summary)

pivot = pd.pivot_table(df, values="revenue", index="region",
                        columns="product", aggfunc="sum", fill_value=0)
print(pivot)

```

OUTPUT

```

region
East      2440
North     3690
South     1400
Name: revenue, dtype: int64

```

	orders	units	revenue
product			
Laptop	3	6	5700
Monitor	1	4	840
Mouse	2	55	990
product	Laptop	Monitor	Mouse
region			
East	1900	0	540
North	2850	840	0
South	950	0	450

Script 14.9 Handling missing data

ch14/s14_09_handling_missing_data.py [pip install numpy](#) [pip install pandas](#)

`isna().sum()` counts gaps; `fillna()` fills them with the mean or a label.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd
```

```
import numpy as np

survey = pd.DataFrame({
    "customer": ["A", "B", "C", "D"],
    "satisfaction": [4, np.nan, 5, 3],
    "city": ["Beirut", "Byblos", None, "Beirut"],
})

print(survey.isna().sum())
survey["satisfaction"] = survey["satisfaction"].fillna(
    survey["satisfaction"].mean())
survey["city"] = survey["city"].fillna("Unknown")
print(survey)
```

OUTPUT

```
customer      0
satisfaction   1
city           1
dtype: int64
   customer  satisfaction    city
0         A           4.0  Beirut
1         B           4.0  Byblos
2         C           5.0  Unknown
3         D           3.0  Beirut
```

Script 14.10 Cleaning text columns and dates

ch14/s14_10_cleaning_text_columns_and_dates.py [pip install pandas](#)

The `.str` accessor cleans text columns; `to_datetime()` converts text to dates and `.dt` extracts parts such as the month name.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
```

```
df["product"] = df["product"].str.strip().str.title()
orders = pd.DataFrame({"date": ["2026-09-01", "2026-09-15", "2026-10-02"]})
orders["date"] = pd.to_datetime(orders["date"])
orders["month"] = orders["date"].dt.month_name()
print(orders)
```

OUTPUT

	date	month
0	2026-09-01	September
1	2026-09-15	September
2	2026-10-02	October

Script 14.11 Exporting results

ch14/s14_11_exporting_results.py [pip install openpyxl](#) [pip install pandas](#)

to_csv() and to_excel() save results for colleagues. Excel output requires openpyxl.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd
```

```
data = {
    "order_id": [101, 102, 103, 104, 105, 106],
    "region": ["North", "South", "North", "East", "South", "East"],
    "product": ["Laptop", "Mouse", "Monitor", "Laptop", "Laptop", "Mouse"],
    "units": [3, 25, 4, 2, 1, 30],
    "unit_price": [950, 18, 210, 950, 950, 18],
}
df = pd.DataFrame(data)
df["revenue"] = df["units"] * df["unit_price"]
summary = df.groupby("product").agg(
    orders=("order_id", "count"),
    units=("units", "sum"),
    revenue=("revenue", "sum"),
)
```

```
summary.to_csv("product_summary.csv")
```

```
summary.to_excel("product_summary.xlsx", sheet_name="Summary")
```

NO SCREEN OUTPUT · CREATES PRODUCT_SUMMARY.CSV, WHICH CONTAINS:

```
product,orders,units,revenue
Laptop,3,6,5700
Monitor,1,4,840
Mouse,2,55,990
```

CHAPTER 15

Visualizing Business Data with Matplotlib

Turning numbers into charts: line, bar, pie and scatter charts, plotting directly from pandas, and a two-chart mini dashboard.

Libraries used: matplotlib, pandas

Script 15.1 Line chart: revenue vs target

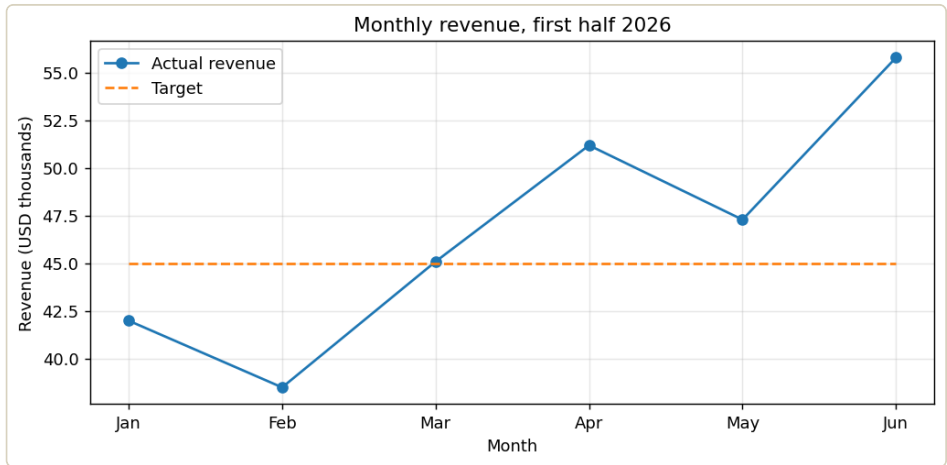
ch15/s15_01_line_chart_revenue_vs_target.py [pip install matplotlib](#)

plot() draws lines; marker, linestyle and label style them. savefig() saves a PNG and show() opens the window.

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
revenue = [42, 38.5, 45.1, 51.2, 47.3, 55.8]      # thousands USD
target = [45] * 6

plt.figure(figsize=(8, 4))
plt.plot(months, revenue, marker="o", label="Actual revenue")
plt.plot(months, target, linestyle="--", label="Target")
plt.title("Monthly revenue, first half 2026")
plt.xlabel("Month")
plt.ylabel("Revenue (USD thousands)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("revenue_trend.png", dpi=150)
plt.show()
```



Script 15.2 Bar chart: sales by region

ch15/s15_02_bar_chart_sales_by_region.py [pip install matplotlib](#)

Sorting the bars from largest to smallest makes comparisons easy; `bar_label()` writes values on the bars.

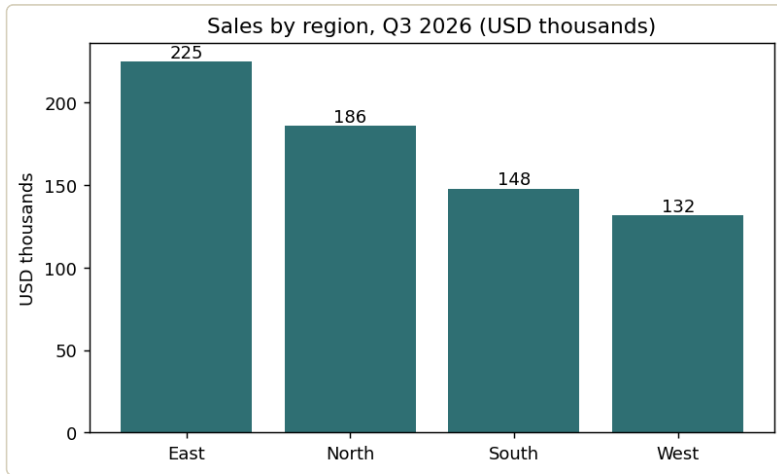
SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import matplotlib.pyplot as plt
```

```
regions = ["North", "South", "East", "West"]
sales = [186, 148, 225, 132]

pairs = sorted(zip(sales, regions), reverse=True)
sales_sorted = [s for s, r in pairs]
regions_sorted = [r for s, r in pairs]

plt.figure(figsize=(7, 4))
bars = plt.bar(regions_sorted, sales_sorted, color="#2f6f73")
plt.bar_label(bars, fmt="%d")
plt.title("Sales by region, Q3 2026 (USD thousands)")
plt.ylabel("USD thousands")
plt.show()
```



Script 15.3 Pie chart: revenue by channel

ch15/s15_03_pie_chart_revenue_by_channel.py [pip install matplotlib](#)

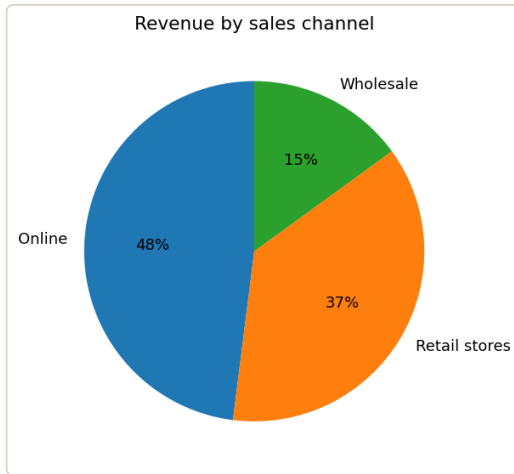
autopct prints percentages on the slices. Use pie charts sparingly, and only for a few parts of one whole.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import matplotlib.pyplot as plt
```

```
channels = ["Online", "Retail stores", "Wholesale"]
share = [48, 37, 15]

plt.figure(figsize=(5, 5))
plt.pie(share, labels=channels, autopct="%1.0f%%", startangle=90)
plt.title("Revenue by sales channel")
plt.show()
```



Script 15.4 Scatter plot: ad spend vs sales

ch15/s15_04_scatter_plot_ad_spend_vs_sales.py [pip install matplotlib](#)

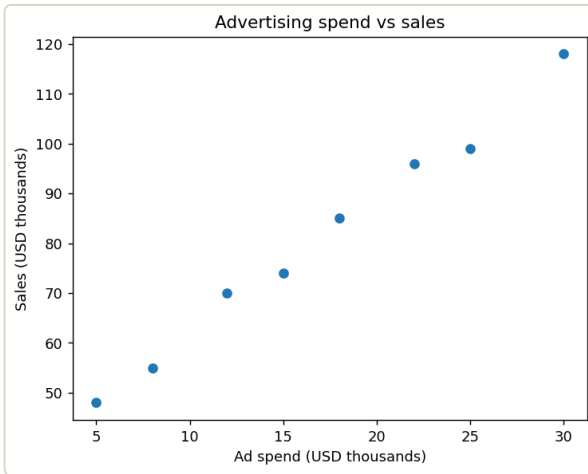
Each dot is one observation, which shows the relationship between two numeric variables.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import matplotlib.pyplot as plt
```

```
ad_spend = [5, 8, 12, 15, 18, 22, 25, 30]
sales = [48, 55, 70, 74, 85, 96, 99, 118]
```

```
plt.scatter(ad_spend, sales)
plt.title("Advertising spend vs sales")
plt.xlabel("Ad spend (USD thousands)")
plt.ylabel("Sales (USD thousands)")
plt.show()
```



Script 15.5 Plotting directly from pandas

ch15/s15_05_plotting_directly_from_pandas.py

[pip install matplotlib](#)

[pip install pandas](#)

`df.plot()` draws charts straight from a `DataFrame`; `stacked=True` stacks the bars.

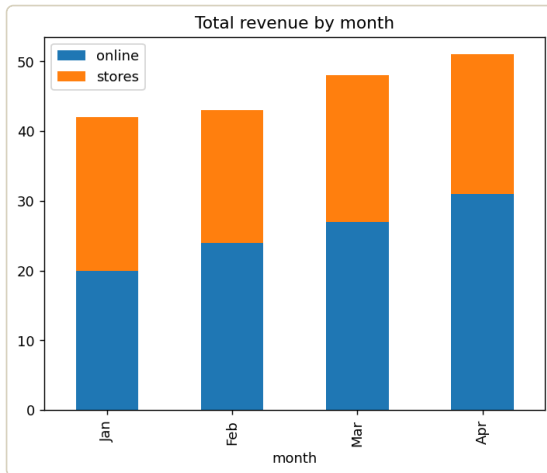
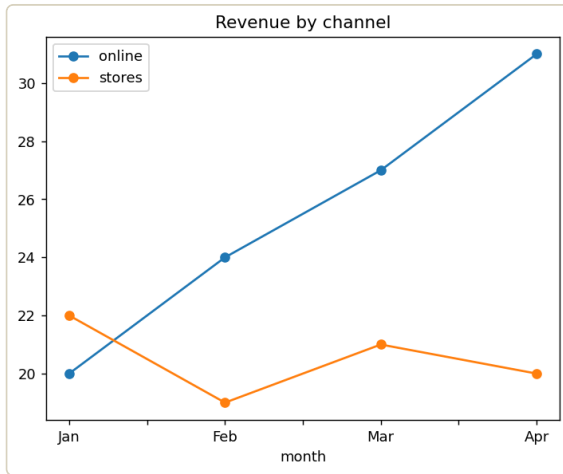
SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar", "Apr"],
    "online": [20, 24, 27, 31],
    "stores": [22, 19, 21, 20],
}).set_index("month")
```

```
df.plot(kind="line", marker="o", title="Revenue by channel")
df.plot(kind="bar", stacked=True, title="Total revenue by month")
plt.show()
```



Script 15.6 Subplots: a mini dashboard

ch15/s15_06_subplots_a_mini_dashboard.py [pip install matplotlib](#)

`plt.subplots(1, 2)` creates two charts side by side; each axes object is drawn separately.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import matplotlib.pyplot as plt

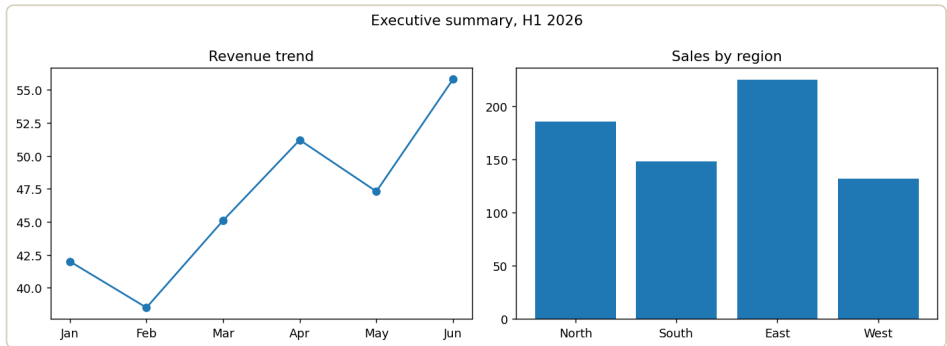
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
revenue = [42, 38.5, 45.1, 51.2, 47.3, 55.8]
regions = ["North", "South", "East", "West"]
sales = [186, 148, 225, 132]
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(months, revenue, marker="o")
axes[0].set_title("Revenue trend")

axes[1].bar(regions, sales[:4])
axes[1].set_title("Sales by region")

fig.suptitle("Executive summary, H1 2026")
fig.tight_layout()
plt.show()
```



CHAPTER 16

Capstone Project: Inventory and Sales Manager

A complete menu-driven application that combines classes, files (JSON and CSV), exceptions, match and pandas. The five parts below form one program, `inventory_manager.py`.

Libraries used: `csv`, `datetime`, `json`, `openpyxl`, `pandas`, `pathlib`

Save Parts 1–5 one after another in a single file named `inventory_manager.py` (already assembled in the download) and run it. It creates `inventory.json` and `sales_log.csv` next to itself, and option 6 writes `sales_report.xlsx` (needs `pandas` + `openpyxl`).

Script 16.1 Part 1: settings and the Product class

ch16/inventory_manager.py

Standard library · csv

Standard library · datetime

Standard library · json

Standard library · pathlib

Imports, file locations and the Product class, which validates its data, flags low stock and converts itself to a dictionary for saving.

```
import csv
import json
from datetime import datetime
from pathlib import Path

INVENTORY_FILE = Path("inventory.json")
SALES_FILE = Path("sales_log.csv")
REORDER_LEVEL = 5

class Product:
    """A single stock item."""

    def __init__(self, sku, name, price, stock=0):
        if price < 0 or stock < 0:
            raise ValueError("Price and stock cannot be negative.")
        self.sku = sku.upper()
        self.name = name
        self.price = float(price)
        self.stock = int(stock)

    def needs_reorder(self):
        return self.stock < REORDER_LEVEL

    def to_dict(self):
        return {"sku": self.sku, "name": self.name,
                "price": self.price, "stock": self.stock}

    def __str__(self):
        flag = " LOW STOCK" if self.needs_reorder() else ""
        return (f"{self.sku:<10}{self.name:<24}"
                f"{self.price:>9.2f}{self.stock:>7}}{flag}")
```

Script 16.2 Part 2: the Inventory class

`ch16/inventory_manager.py``Standard library · csv``Standard library · datetime``Standard library · json``Standard library · pathlib`

Loads and saves products as JSON, adds, restocks and sells items with validation, logs every sale to CSV, and produces reports.

```

class Inventory:
    """Manages all products and records sales."""

    def __init__(self):
        self.products = {}
        self.load()

    # ----- persistence -----
    def load(self):
        if INVENTORY_FILE.exists():
            with open(INVENTORY_FILE) as f:
                for item in json.load(f):
                    p = Product(**item)
                    self.products[p.sku] = p

    def save(self):
        with open(INVENTORY_FILE, "w") as f:
            json.dump([p.to_dict() for p in self.products.values()], f,
                      indent=2)

    # ----- operations -----
    def add_product(self, sku, name, price, stock):
        sku = sku.upper()
        if sku in self.products:
            raise ValueError(f"SKU {sku} already exists.")
        self.products[sku] = Product(sku, name, price, stock)
        self.save()

    def restock(self, sku, units):
        product = self._get(sku)
        if units <= 0:
            raise ValueError("Restock quantity must be positive.")
        product.stock += units
        self.save()

    def sell(self, sku, units):
        product = self._get(sku)
        if units <= 0:
            raise ValueError("Sale quantity must be positive.")
        if units > product.stock:
            raise ValueError(f"Only {product.stock} units of "
                             f"{product.name} in stock.")
        product.stock -= units
        self.save()
        self._log_sale(product, units)
        return units * product.price

    def _get(self, sku):
        try:
            return self.products[sku.upper()]
        except KeyError:

```

```

        raise KeyError(f"No product with SKU {sku.upper()}.") from None

def _log_sale(self, product, units):
    new_file = not SALES_FILE.exists()
    with open(SALES_FILE, "a", newline="") as f:
        writer = csv.writer(f)
        if new_file:
            writer.writerow(["timestamp", "sku", "name", "units",
                             "unit_price", "revenue"])
        writer.writerow([datetime.now().isoformat(timespec="seconds"),
                         product.sku, product.name, units,
                         product.price, units * product.price])

# ----- reporting -----
def list_products(self):
    print(f"{'SKU':<10}{ 'Name':<24}{ 'Price':>9}{ 'Stock':>7}")
    print("-" * 50)
    for p in sorted(self.products.values(), key=lambda p: p.sku):
        print(p)

def low_stock(self):
    return [p for p in self.products.values() if p.needs_reorder()]

def stock_value(self):
    return sum(p.price * p.stock for p in self.products.values())

```

Script 16.3 Part 3: helper functions for safe input

ch16/inventory_manager.py **Built-in only · nothing to install**

Re-ask until the user enters non-empty text or a valid, non-negative number.

```

def ask_text(prompt):
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print(" This field cannot be empty.")

def ask_number(prompt, kind=float):
    while True:
        try:
            value = kind(input(prompt))
            if value < 0:
                print(" Please enter a non-negative number.")
                continue
            return value
        except ValueError:
            print(" Please enter a valid number.")

```

Script 16.4 Part 4: the sales report with pandasch16/inventory_manager.py `pip install openpyxl` `pip install pandas`

Groups the sales log by product, prints totals, and exports sales_report.xlsx. pandas is imported only when the report is requested.

```
def sales_report():
    if not SALES_FILE.exists():
        print("No sales recorded yet.")
        return
    import pandas as pd
    df = pd.read_csv(SALES_FILE)
    report = (df.groupby(["sku", "name"])
              .agg(units=("units", "sum"), revenue=("revenue", "sum"))
              .sort_values("revenue", ascending=False))
    print(report.to_string())
    print(f"\nTotal revenue: {df['revenue'].sum():.2f}")
    report.to_excel("sales_report.xlsx")
    print("Report saved to sales_report.xlsx")
```

Script 16.5 **Part 5: the main menu**

ch16/inventory_manager.py [Standard library · csv](#) [Standard library · datetime](#) [Standard library · json](#)
[Standard library · pathlib](#) [pip install openpyxl](#) [pip install pandas](#) [Python 3.10+](#)

A while loop shows the menu; match routes each choice; errors are caught and shown without crashing. `if __name__ == "__main__":` starts the program.

```

MENU = """
===== CEDAR TRADING CO. INVENTORY =====
1. List products
2. Add product
3. Restock product
4. Record sale
5. Low-stock alert
6. Sales report
7. Exit
"""

def main():
    inv = Inventory()
    while True:
        print(MENU)
        choice = input("Choose an option: ").strip()
        try:
            match choice:
                case "1":
                    inv.list_products()
                    print(f"Total stock value: {inv.stock_value():.2f}")
                case "2":
                    inv.add_product(ask_text("SKU: "), ask_text("Name: "),
                                   ask_number("Price: "),
                                   ask_number("Stock: ", int))
                    print("Product added.")
                case "3":
                    inv.restock(ask_text("SKU: "),
                               ask_number("Units to add: ", int))
                    print("Stock updated.")
                case "4":
                    revenue = inv.sell(ask_text("SKU: "),
                                       ask_number("Units sold: ", int))
                    print(f"Sale recorded. Revenue: {revenue:.2f}")
                case "5":
                    low = inv.low_stock()
                    if low:
                        for p in low:
                            print(f"Reorder {p.name} (only {p.stock} left)")
                    else:
                        print("All products above reorder level.")
                case "6":
                    sales_report()
                case "7":
                    print("Goodbye.")
                    break
                case _:
                    print("Invalid option.")
        except (ValueError, KeyError) as e:
            print("Error:", e)

```

```
if __name__ == "__main__":  
    main()
```

SAMPLE SESSION · RUN IN AN EMPTY FOLDER · **HIGHLIGHTED** = TYPED BY THE USER

===== CEDAR TRADING CO. INVENTORY =====

1. List products
2. Add product
3. Restock product
4. Record sale
5. Low-stock alert
6. Sales report
7. Exit

Choose an option: **2**

SKU: **Lap-0932**

Name: **Business Laptop 14**

Price: **949**

Stock: **6**

Product added.

... (menu shown again) ...

Choose an option: **4**

SKU: **LAP-0932**

Units sold: **9**

Error: Only 6 units of Business Laptop 14 in stock.

... (menu shown again) ...

Choose an option: **4**

SKU: **LAP-0932**

Units sold: **2**

Sale recorded. Revenue: 1,898.00

... (menu shown again) ...

Choose an option: **5**

Reorder Business Laptop 14 (only 4 left)

... (menu shown again) ...

Choose an option: **1**

SKU	Name	Price	Stock
LAP-0932	Business Laptop 14	949.00	4 LOW STOCK

Total stock value: 3,796.00

... (menu shown again) ...

Choose an option: **6**

sku	name	units	revenue
LAP-0932	Business Laptop 14	2	1898.0

Total revenue: 1,898.00

Report saved to sales_report.xlsx

... (menu shown again) ...

```
Choose an option: 7
Goodbye.
```

APPENDIX C

Answers to Selected Exercises

Model solutions from Appendix C of the book. Try each exercise yourself first, then compare.

Libraries used: pandas

Script C.1 Ch 2, Ex 2: overtime pay

appendix_c_answers/sc_01_overtime_pay.py **Built-in only · nothing to install**

Hours above 40 are paid at $1.5 \times$ the rate; `min()` and `max()` split regular and overtime hours.

```
rate = 18.75
hours = 42
regular = min(hours, 40) * rate
overtime = max(hours - 40, 0) * rate * 1.5
print(regular + overtime)    # 806.25
```

OUTPUT

```
806.25
```

Script C.2 Ch 2, Ex 3: pallets

appendix_c_answers/sc_02_pallets.py **Built-in only · nothing to install**

// gives full pallets of 48 and % the units left over.

```
print(1000 // 48, 1000 % 48)    # 20 full pallets, 40 units left
```

OUTPUT

```
20 40
```

Script C.3 Ch 3, Ex 1: email domain

appendix_c_answers/sc_03_email_domain.py **Built-in only · nothing to install**

Clean the email, then split at @ and take the second part.

```
email = " Sara.Nassar@Company.LB "
clean = email.strip().lower()
domain = clean.split("@")[1]
print(clean, domain)    # sara.nassar@company.lb company.lb
```

OUTPUT

```
sara.nassar@company.lb company.lb
```

Script C.4 Ch 5, Ex 2: shipping fee

```
appendix_c_answers/sc_04_shipping_fee.py
```

 Built-in only · nothing to install

A three-band shipping rule with if-elif-else.

```
total = float(input("Order total: "))
if total >= 100:
    shipping = 0
elif total >= 50:
    shipping = 2
else:
    shipping = 5
print(f"Shipping: ${shipping}")
```

SAMPLE SESSION · HIGHLIGHTED = TYPED BY THE USER

```
Order total: 75
Shipping: $2
```

Script C.5 Ch 6, Ex 3: revenue doubling

```
appendix_c_answers/sc_05_revenue_doubling.py
```

 Built-in only · nothing to install

A while loop grows revenue by 12% a year until it doubles.

```
revenue, years = 500_000, 0
while revenue <= 1_000_000:
    revenue *= 1.12
    years += 1
print(years)    # 7
```

OUTPUT

```
7
```

Script C.6 Ch 6, Ex 5: straight-line depreciation

appendix_c_answers/sc_06_straight_line_depreciation.py

Built-in only · nothing to install

The same depreciation is charged every year, and book value falls until it reaches salvage value.

```
cost, salvage, life = 12_000, 2_000, 5
annual = (cost - salvage) / life
book = cost
for year in range(1, life + 1):
    book -= annual
    print(f"Year {year}: depreciation {annual:,.2f}, book value {book:,.2f}")
```

OUTPUT

```
Year 1: depreciation 2,000.00, book value 10,000.00
Year 2: depreciation 2,000.00, book value 8,000.00
Year 3: depreciation 2,000.00, book value 6,000.00
Year 4: depreciation 2,000.00, book value 4,000.00
Year 5: depreciation 2,000.00, book value 2,000.00
```

Script C.7 Ch 7, Ex 3: remove duplicates, keep order

appendix_c_answers/sc_07_remove_duplicates_keep_order.py

Built-in only · nothing to install

Adds each name only if it is not already in the new list. Unlike set(), this keeps the original order.

```
names = ["Hala", "Omar", "Hala", "Nour", "Omar"]
unique = []
for n in names:
    if n not in unique:
        unique.append(n)
print(unique) # ['Hala', 'Omar', 'Nour']
```

OUTPUT

```
['Hala', 'Omar', 'Nour']
```

Script C.8 Ch 8, Ex 2: count by region

appendix_c_answers/sc_08_count_by_region.py

Built-in only · nothing to install

The dictionary counting pattern from Script 8.4.

```
regions = ["North", "South", "North", "East", "North"]
counts = {}
for r in regions:
    counts[r] = counts.get(r, 0) + 1
print(counts) # {'North': 3, 'South': 1, 'East': 1}
```

OUTPUT

```
{'North': 3, 'South': 1, 'East': 1}
```

Script C.9 Ch 9, Ex 3: simple email check

appendix_c_answers/sc_09_simple_email_check.py **Built-in only · nothing to install**

Exactly one @, something before it, and a dot after it.

```
def is_valid_email(text):
    if text.count("@") != 1:
        return False
    local, domain = text.split("@")
    return bool(local) and "." in domain

print(is_valid_email("maya@example.com"))      # True
print(is_valid_email("maya.example.com"))      # False
```

OUTPUT

```
True
False
```

Script C.10 Ch 13, Ex 3: SavingsAccount

appendix_c_answers/sc_10_savingsaccount.py **Built-in only · nothing to install**

Inherits from BankAccount (Script 13.3) and adds interest through the existing deposit() method.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self._balance = balance
        self._transactions = []

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive.")
        self._balance += amount
        self._transactions.append(("deposit", amount))

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("Insufficient funds.")
        self._balance -= amount
        self._transactions.append(("withdrawal", amount))

    @property
    def balance(self):
        return self._balance

    def statement(self):
        for kind, amount in self._transactions:
            print(f"{kind:<12}{amount:>10,.2f}")
        print(f"{'balance':<12}{self._balance:>10,.2f}")
```

```
class SavingsAccount(BankAccount):
    def __init__(self, owner, balance=0.0, rate=0.03):
        super().__init__(owner, balance)
        self.rate = rate

    def add_interest(self):
        self.deposit(self.balance * self.rate)

savings = SavingsAccount("Maya Haddad", 1000)
savings.add_interest()
print(savings.balance)
```

OUTPUT

1030.0

One demonstration line was added at the end so the script prints a visible result.

Script C.11 Ch 14, Ex 3: department summaryappendix_c_answers/sc_11_department_summary.py [pip install pandas](#)

groupby() with named aggregations: headcount and average salary per department.

SETUP · ADDED SO THIS SCRIPT RUNS ON ITS OWN

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Hala", "Omar", "Nour", "Karim", "Lea"],
    "department": ["Finance", "IT", "Finance", "IT", "HR"],
    "salary": [3200, 2750, 4100, 3900, 2600],
})
```

```
summary = df.groupby("department").agg(
    headcount=("name", "count"),
    avg_salary=("salary", "mean"),
)
print(summary)
```

OUTPUT

	headcount	avg_salary
department		
Finance	2	3650.0
HR	1	2600.0
IT	2	3325.0

Troubleshooting common problems

You see...	What it means / what to do
<code>ModuleNotFoundError: No module named 'pandas'</code>	The library is not installed for the Python you are running. Use <code>py -m pip install pandas</code> (Windows) or <code>python3 -m pip install pandas</code> . The <code>-m pip</code> form installs into the same Python that runs your script.
<code>'pip' is not recognized...</code>	Python is not on PATH. Use <code>py -m pip ...</code> , or reinstall Python and tick “Add python.exe to PATH”.
<code>ImportError: Missing optional dependency 'openpyxl'</code>	pandas needs openpyxl to read or write .xlsx files: <code>pip install openpyxl</code> .
<code>SyntaxError</code> on a <code>match</code> line	Your Python is older than 3.10. Upgrade Python, or use <code>if/elif</code> instead.
<code>FileNotFoundError: ... 'sales.csv'</code>	The script cannot see the data file. Put the file in the same folder and run the script from that folder (<code>cd</code> into it first).
<code>IndentationError</code>	Mixed or missing indentation. Use 4 spaces per level and never mix tabs and spaces.
<code>NameError: name 'df' is not defined</code>	The variable was created in an earlier script. Include the Setup block, or run the earlier cell first in Jupyter.
<code>TypeError: can only concatenate str...</code>	You joined text and a number with <code>+</code> . Use an f-string or <code>str()</code> .
<code>ValueError: invalid literal for int()</code>	Text could not become a number (for example “ten” or “12.5” given to <code>int()</code>). Validate with <code>try/except</code> (Scripts 12.2 and 12.5).
Chart window never appears	You forgot <code>plt.show()</code> . In Jupyter charts appear automatically. You can also save with <code>plt.savefig("chart.png")</code> .

Reading any error: start at the **last line** (the error type and message), then look at the line number just above it. Change one thing at a time and run the script again.